DPGS: Differentially Private Graph Synthesis via Graph Summarization

Xun Ran[1], Qingqing Ye[1,*], Jian Lou[2], Puning Zhao[2]
[1]The Hong Kong Polytechnic University, [2]Sun Yat-sen University
{qi-xun.ran@connect.polyu.hk, qqing.ye@polyu.edu.hk,
louj5@mail.sysu.edu.cn, zhaopn@mail.sysu.edu.cn}

***Abstract*—Many real-world systems, such as social and email networks, can be represented as graphs. Analyzing these structures is crucial for various applications but raises significant privacy concerns. Differential privacy (DP) provides a formal framework for protecting sensitive graph data, ensuring minimal impact from changing individual nodes or edges on analysis results. A canonical approach in this context is to generate a synthetic graph under DP constraints, while preserving the characteristics of the original graph, to support downstream graph analysis. However, existing graph synthesis solutions either perturb the adjacency matrix directly, leading to high noise sensitivity, or use alternative encoding methods that suffer from significant information loss. To address this, we propose DPGS, a novel framework for Differentially Private Graph Synthesis via graph summarization. By extracting a summarized representation of the original graph, DPGS encodes structural patterns before applying DP, enhancing noise resilience while preserving key properties. Experiments on real-world datasets show that DPGS outperforms existing methods in maintaining structural features under privacy constraints, leading to effective graph analysis with enhanced utility.**

***Index Terms*—Data privacy, Graph data, Differential privacy, Graph neural networks**

I. Introduction

A wide range of complex systems can be modeled as graphs, including domains such as social interactions [1], scholarly communication networks [2], and recommendation platforms [3]. The study of graph-structured data constitutes a fundamental component of numerous scientific and industrial applications [4]. For example, Amazon recommends products by analyzing co-purchase networks, where users (i.e., nodes) are linked by shared purchases (i.e., edges) [5]. However, the sensitive nature of graph data raises significant privacy concerns, making direct analysis without appropriate safeguards impractical.

A traditional approach to privacy preservation in graph analysis is anonymization, which removes identifying information from nodes [6], [7]. However, prior studies have demonstrated that anonymized graphs remain vulnerable to de-anonymization attacks when adversaries possess auxiliary information [8], [9].

As the gold standard in the privacy community, differential privacy (DP) [10], [11], [12], [13] has been widely adopted to protect graph data [14], [15]. The central principle of DP is to bound the influence of any single node or edge on the output of an analysis. Existing research on differentially private graph analysis has primarily focused on task-specific algorithms, such as releasing degree distributions [16], counting subgraphs [17], and community detection [18]. However, these approaches are inherently task-dependent, requiring new DP mechanisms to be designed for each additional task.

In contrast, releasing a differentially private synthetic graph that preserves the semantic structure of the original graph provides a general task-agnostic solution, enabling a wide range of downstream graph analysis tasks.

Despite these advantages, designing effective methods for accurate synthetic graph generation remains challenging. The core difficulty lies in the inherent trade-off between information retention and noise resilience. Fine-grained approaches, such as directly perturbing the adjacency matrix, preserve rich structural details but are highly sensitive to noise, leading to substantial accuracy degradation under stringent privacy budgets. In contrast, coarse-grained or aggregated approaches, including clustering and hierarchical models, are more robust to noise but inevitably sacrifice fine-grained information, thereby limiting reconstruction fidelity.

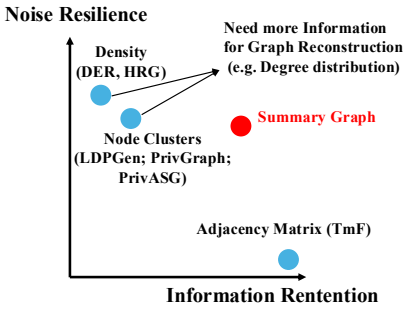


Fig. 1: Comparison of Graph Encoding Methods

Figure 1 summarizes existing works along these two dimensions. Approaches such as Top-m Filter (TmF) [19], which perturb the adjacency matrix directly, lie at the high-information but low-resilience end of the spectrum. Density-based Exploration and Reconstruction (DER) [20] improves noise resilience by aggregating dense regions via quadtree-based partitioning, but it struggles on sparse graphs and incurs high computational overhead. Hierarchical Random Graph (HRG) [21] further enhances noise robustness through hierarchical encoding, albeit at the cost of structural distortion and additional modeling complexity. Node clustering methods, such as LDPGen [22], aggregate graph information prior to noise injection, improving robustness but introducing approximation errors that limit reconstruction accuracy. More

*Corresponding author.

recent approaches, such as PrivGraph [23], additionally rely on external statistics (e.g., degree distributions), which may introduce further privacy risks and reconstruction bias. PrivAGS [24] extends privacy protection to both graph structure and node features; however, its structure encoding follows that of PrivGraph and remains decoupled from feature information, requiring auxiliary inputs for reconstruction. Moreover, it typically protects only a single node feature, limiting its effectiveness for graphs with high-dimensional feature spaces.

Our Proposal. This work proposes DPGS, a graph summarization-based approach that balances noise resilience and structural preservation. Graph summarization represents a graph using supernodes and superedges, simplifying its structure while retaining essential connectivity patterns [25]. An example of this transformation is illustrated in Figure 2, where an input graph (Figure 2(a)) is transformed into a summary graph (Figure 2(b)). Perturbing this compressed representation reduces dimensionality and mitigates noise amplification, while preserving sufficient structural information for high-quality synthetic graph generation. Given its broad adoption in social network analysis, biological networks, and recommender systems, graph summarization provides a natural foundation for privacy-preserving graph synthesis.

Representing a graph at a higher level of abstraction by summarizing its structural patterns, graph summarization allows us to introduce less perturbation to satisfy DP. Specifically, by grouping nodes into supernodes and aggregating connections into superedges, summarization reduces the total number of elements (compared to the original adjacency matrix), which directly lowers the function's global sensitivity—the maximum change caused by modifying a single edge or node. This reduction enables the addition of less noise to achieve the same level of privacy.

Moreover, by capturing both intra- and inter-group relationships, the summary retains essential connectivity patterns—such as community structures, dense regions, and cross-group links—so that, even with fine-grained details abstracted, it still supports accurate reconstruction of graph properties and synthetic graph generation.

Designing a method that balances utility and privacy remains challenging. Many existing graph summarization approaches, such as random search [25], [23], operate solely on structural signals and struggle to incorporate node features effectively. In contrast, GNNs naturally integrate both structure and features through message aggregation, enabling more meaningful node representations and better initial partitions. However, this advantage comes with privacy risks, as aggregation induces data dependencies that can amplify noise under DP constraints [26]. To address this trade-off, we propose a hybrid approach: we first leverage GNNs to obtain feature-aware initial clusterings, and then refine them via random search. The GNN-induced noise is mitigated through a spectral clustering module, while the refinement phase employs the Exponential Mechanism to reduce noise accumulation and accelerate convergence.

Following this idea, DPGS first extracts a summarized representation of the graph, capturing both local and global structures. We then perturb this summary instead of the raw adjacency matrix, allowing for a more stable and noise-resilient encoding. To ensure fidelity, DPGS separately processes dense substructures and sparse interconnections, applying different perturbation strategies tailored to each. Finally, a post-processing step refines the synthetic graph, ensuring consistency and enhancing utility. In summary, our main contributions are threefold:

- We propose DPGS, a novel differentially private synthetic graph generation framework that leverages graph summarization techniques to improve noise resilience while preserving structural integrity.
- We propose a private selection strategy that uses the exponential mechanism to achieve differentially private graph summarization while balancing utility and privacy.
- We conduct extensive experiments across multiple datasets and evaluation metrics, which validate the superiority of DPGS over existing approaches.

The remainder of this paper is organized as follows. In Sections 2 and 3, we introduce the preliminaries and the problem statement. Section 4 presents the proposed scheme and Section 5 gives the theoretical analysis. The experimental results are reported and analyzed in Section 6. Section 7 reviews related work. In Section 8, the conclusion is drawn.

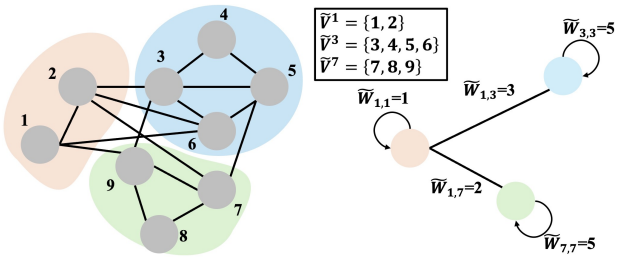
II. Preliminaries

In this section, we introduce preliminaries of our study. We first present graph summarization techniques, which serve as the building block of this work, followed by the background of differential privacy and graph neural networks.

A. Graph Summarization

Input graph: Consider an undirected graph $G = (V, E, \boldsymbol{W}, \boldsymbol{X})$ where $V = \{v_1, v_2, \ldots, v_p\}$ is the set of p nodes, $E \subseteq V \times V$ is the edge set and $\boldsymbol{W}$ is the adjacency (weight) matrix. We assume that G is undirected without self-loops: $\boldsymbol{W}_{i,j} > 0$ if $(i,j) \in E$, and $\boldsymbol{W}_{i,j} = 0$ if $(i,j) \notin E$. Finally, $\boldsymbol{X} \in \mathbb{R}^{p \times n} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_p]^T$ is the feature matrix, where each row vector $\boldsymbol{x}_i \in \mathbb{R}^n$ is the feature vector associated with one of p nodes in the graph G. A graph can be conveniently represented by Laplacian matrix $\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{W}$, where $\boldsymbol{D} = \mathrm{diag}(\boldsymbol{W}\mathbf{1})$ is the degree matrix and $\mathbf{1} \in \mathbb{R}^p$ denotes the all-one vector. The graph Laplacian is well established as a foundation for embedding, clustering, and semi-supervised learning. Consequently, Laplacian-based representations provide particularly suitable building blocks for graph algorithms. We refer to the nodes and edges in G as subnodes and subedges, respectively, in order to distinguish them from the supernodes and superedges that appear in the summary graphs discussed below.

Summary graph: A summary graph $\tilde{G} = (\tilde{V}, \tilde{E}, \tilde{\boldsymbol{W}}, \tilde{\boldsymbol{X}})$ of an original graph $G = (V, E, \boldsymbol{W}, \boldsymbol{X})$ is defined by a set of supernodes $\tilde{V}$, a set of superedges $\tilde{E}$, a weight matrix $\tilde{\boldsymbol{W}}$, and a transformed feature matrix $\tilde{\boldsymbol{X}}$. Each superedge $(\tilde{V}^u, \tilde{V}^v) \in \tilde{E}$ connects two supernodes $\tilde{V}^u, \tilde{V}^v \in \tilde{V}$. When



(a) Input Graph G (b) Summary Graph $\tilde{G}$

Fig. 2: An illustration of graph summarization. Each subnode in the input graph G is assigned to exactly one supernode $\tilde{V}^u$ in the summary graph $\tilde{G}$. The weight of each superedge $\tilde{\boldsymbol{W}}_{u,v}$ in $\tilde{G}$ represents the number of subedges connecting the two associated supernodes.

$u = v$, the edge $(\tilde{V}^v, \tilde{V}^v)$ represents a self-loop at supernode $\tilde{V}^v$. Let $\Pi_{\tilde{V}} := \{(\tilde{V}^u, \tilde{V}^v) : \tilde{V}^u, \tilde{V}^v \in \tilde{V}\}$ denote the set of all unordered pairs of supernodes, and $\tilde{E} \subseteq \Pi_{\tilde{V}}$. For each superedge $(\tilde{V}^u, \tilde{V}^v) \in \tilde{E}$, the corresponding entry of the weight matrix is defined by the number of subedges between the two supernodes, i.e., $\tilde{\boldsymbol{W}}_{u,v} := \left|\{(i,j) \in E : i \in \tilde{V}^u, j \in \tilde{V}^v\}\right|$. An illustrative example of a summary graph is provided in Figure 2.

Reconstructed graph: Given a summary graph $\tilde{G}$, we can reconstruct a graph $\hat{G} = (\hat{V}, \hat{E}, \hat{\boldsymbol{W}}, \hat{\boldsymbol{X}})$ following the conventional approach in [27], [28]. The set of subnodes $\hat{V}$ is obtained by taking the union of all supernodes in $\tilde{V}$, while the reconstructed edge set $\hat{E}$ includes all inter- and intra-supernode connections, defined as $\hat{E} := \{(u, v) \in \hat{V} \times \hat{V} : u \neq v,\ (\tilde{V}^u, \tilde{V}^v) \in \tilde{E}\}$. The reconstructed adjacency weights can be computed as: $\hat{\boldsymbol{W}}_{u,v} = \frac{\tilde{\boldsymbol{W}}_{u,v}}{|\Pi_{\tilde{V}^u, \tilde{V}^v}|}$, where $\Pi_{\tilde{V}^u, \tilde{V}^v} := \{(u, v) : u \neq v,\ u \in \tilde{V}^u,\ v \in \tilde{V}^v\}$ denotes the set of all possible subedges between the two corresponding supernodes.

B. Differential Privacy

The standard definition of (ε, δ)-differential privacy [29] bounds the change in output distributions of a randomized mechanism between any pair of neighboring inputs by a multiplicative privacy loss bounded by ε and an additive term δ. Different privacy notions arise from different choices of neighboring relations. Unlike classical edge-level privacy, which only protects the presence or absence of individual edges, the setting considered here additionally accounts for node feature information, which is essential for graph summarization as both the summary and reconstruction depend jointly on structural connectivity and node features.

To formalize this privacy requirement, we adopt a neighboring relation that captures a minimal yet meaningful difference between input graphs as follows.

Definition 1 (Feature-Edge Neighboring Graphs)**.** *Let $G = (V, E, \boldsymbol{W}, \boldsymbol{X})$ and $G' = (V, E', \boldsymbol{W}', \boldsymbol{X}')$ be two graphs defined on the same node set V. We say that G and G' are* feature-edge neighboring graphs*, denoted by $G \sim G'$, if the following two conditions hold:*

- *(*Edge difference*) The graphs differ in at most one edge, i.e., $|E \triangle E'| \leq 1$;*
- *(*Feature difference*) There exists at most one node $v \in V$ such that $\boldsymbol{x}_v \neq \boldsymbol{x}'_v$, while $\boldsymbol{x}_u = \boldsymbol{x}'_u$ for all $u \neq v$.*

This relation captures a worst-case change in local node features together with a single structural difference, providing stronger protection than edge-level privacy while remaining less restrictive than node-level privacy. This neighboring definition induces the notion of differential privacy used throughout this paper as follows.

Definition 2 ((ε, δ)-Feature-Edge Differential Privacy)**.** *A randomized mechanism $\mathcal{M}$ satisfies* (ε, δ)-feature-edge differential privacy *if for any two feature-edge neighboring graphs $G \sim G'$ and any measurable set $\mathcal{Z} \subseteq \mathrm{Range}(\mathcal{M})$, it holds that*

$$\mathsf{Pr}\left[\mathcal{M}(G) \in \mathcal{Z}\right] \leq e^{\varepsilon} \mathsf{Pr}\left[\mathcal{M}(G') \in \mathcal{Z}\right] + \delta.$$

Gaussian Mechanism. The Gaussian mechanism (GM) achieves (ε, δ)-differential privacy by adding Gaussian noise calibrated to the sensitivity of the query function. By Definition 1, the global sensitivity Δ defined over any two neighboring graphs $G \sim G'$ is

$$\Delta = \max_{G \sim G'} \left\| f(G) - f(G') \right\|_2,$$

where f is a query function on graphs. Then the Gaussian mechanism $\mathcal{A}$ releases

$$\mathcal{A}_f(G) = f(G) + \mathcal{N}\left(0, \sigma^2 \Delta^2\right),$$

where $\sigma \geq \sqrt{2\ln(1.25/\delta)}/\varepsilon$ ensures (ε, δ)-*feature-edge differential privacy*. For vector-valued outputs, independent Gaussian noise is added to each component.

Exponential Mechanism. The Gaussian mechanism is applicable when the output of a function f is a real-valued vector. In contrast, the Exponential Mechanism (EM) [30] is designed for scenarios where the output is selected from a finite set. EM selects outputs with probabilities proportional to the exponential of their quality scores, favoring more accurate outcomes. Given an input dataset G, EM samples an output $o \in \mathcal{O}$ according to a predefined quality function $q(G, o)$, which evaluates how suitable o is given the input G. This mechanism requires careful design of the quality function q, which maps each pair (G, o) to a real-valued utility score. The privacy analysis of EM depends on the global sensitivity of q, defined as: $\Delta_q = \max_o \max_{G \sim G'} |q(G, o) - q(G', o)|$. The Exponential Mechanism $\mathcal{A}$ satisfies (ε, δ)-*feature-edge differential privacy* by sampling o according to

$$\mathsf{Pr}\left[\mathcal{A}_q(G) = o\right] = \frac{\exp\left(\frac{\varepsilon}{2\Delta_q} q(G, o)\right)}{\sum_{o' \in \mathcal{O}} \exp\left(\frac{\varepsilon}{2\Delta_q} q(G, o')\right)}.$$

Composition Properties of (ε, δ)-Feature-Edge Differential Privacy. The following properties describe how (ε, δ)-feature-edge differential privacy behaves under the *feature-edge neighboring* relation when multiple mechanisms are composed, enabling the design of complex private graph algorithms from modular components.

1) **Sequential Composition.** Let $\{\mathcal{A}_1, \ldots, \mathcal{A}_k\}$ be a sequence of randomized mechanisms applied to the same

input graph G. If each $\mathcal{A}_i$ satisfies $(\varepsilon_i, \delta_i)$ *-feature-edge differential privacy*, then their composition satisfies $(\sum_{i=1}^{k} \varepsilon_i, \sum_{i=1}^{k} \delta_i)$-feature-edge differential privacy.

2) **Parallel Composition.** Suppose the input graph is partitioned into k disjoint components (e.g., disjoint node or edge subsets), and each component is processed independently by a mechanism satisfying $(\varepsilon_i, \delta_i)$-feature-edge differential privacy. Then the overall mechanism satisfies $(\max_i \varepsilon_i, \max_i \delta_i)$-feature-edge differential privacy.
3) **Post-processing Invariance.** Any data-independent transformation of the output of a differentially private mechanism does not weaken the privacy guarantee. Specifically, if a mechanism $\mathcal{A}$ satisfies (ε, δ)-feature-edge differential privacy, then for any (possibly randomized) function g, the output $g(\mathcal{A}(G))$ also satisfies (ε, δ)-feature-edge differential privacy.

C. Graph Neural Networks

Given a graph $G = (V, E, \boldsymbol{W}, \boldsymbol{X})$, an r-layer GNN is a parametric function that can be represented by the following operations:

$$\boldsymbol{H} = \text{GNN}(\boldsymbol{X}, \boldsymbol{W}; \boldsymbol{\Theta}_{\text{GNN}}),$$

where $\boldsymbol{H} \in \mathbb{R}^{p \times d'}$ is the node representations. GNNs can effectively capture local topological and feature-based patterns. The resulting representations $\boldsymbol{H}$ are then used to compute cluster assignments, which serve as the foundation for downstream summary construction and noise injection. This not only improves the quality of the summary graph but also ensures that important graph semantics are preserved in the differentially private synthetic graph generation pipeline.

III. Problem Statement

A common approach to privacy-preserving graph analysis is to design dedicated DP mechanisms for specific tasks, such as community discovery and node classification [21]. However, many tasks involve iterative graph queries, each requiring the allocation of a portion of the privacy budget. This becomes unsustainable as the number of queries grows, since the accumulation of privacy budget consumption eventually depletes the total budget, leading to poor utility in final results.

To overcome this limitation, we adopt an alternative paradigm. Instead of designing DP mechanisms separately for each analysis task or allocating privacy budgets to excessive queries, we focus on generating a differentially private *synthetic graph* that retains the structural properties of the original graph while satisfying DP. By doing so, any downstream graph analysis task can be considered as post-processing, without consuming additional privacy budget. This enables a more scalable and practical solution for privacy-preserving graph analytics.

Threat Model. In differentially private graph synthesis, we consider an adversary with unrestricted access to the released synthetic graph who attempts to infer whether a particular edge exists in the source graph, or whether the feature vector of a particular node takes some specific value. This threat model is captured by our feature-edge neighboring relation in Definition 1. For example, given a synthetic social network, the adversary may try to decide whether two users were connected in the original data, or whether a user exhibits a sensitive attribute encoded in the node features.

Given a graph G, our objective is to generate a synthetic graph $\hat{G}$ that preserves key structural properties of G while satisfying feature-edge DP. Instead of directly perturbing the adjacency matrix, we leverage the *graph summarization* technique to encode structural information at a higher level, enhancing noise resilience and privacy protection. To validate the superiority of our solution, we will follow prior work [20], [15], [19] and evaluate the similarity between $\hat{G}$ and G across five metrics widely used in graph analytics, including community structure, node attributes, degree distribution, path-based connectivity, and topology.

IV. DPGS: Synthetic Graph Generation with DP

In this section, we present a synthetic graph generation framework, DPGS, that achieves *differential privacy* through *graph summarization*, avoiding the need to directly perturb the raw graph structure. We will first introduce an overview of DPGS in Section IV-A, with detailed implementation from Sections IV-B to IV-D.

A. An Overview of DPGS

DPGS consists of three stages: **structure summarization** and **feature summarization**, followed by **graph reconstruction**. The key idea lies in summarizing information at a higher level of abstraction, which inherently reduces sensitivity and enables accurate reconstruction under stronger privacy guarantees. Figure 3 shows the workflow of DPGS.

Structure Summarization. To privately capture structural patterns, we design a GNN-based spectral clustering algorithm with DP guarantees. Unlike traditional methods that cluster solely based on structure, we incorporate node features to enhance clustering quality, leveraging the homophily property in real-world networks. To further improve assignment accuracy without exhausting the privacy budget, we introduce a *refinement step using the exponential mechanism* on a small set of uncertain nodes, thereby concentrating the privacy cost where it is most beneficial. The output is a noisy but informative *summary edge matrix* $\bar{\boldsymbol{W}}$, representing the connectivity between supernodes after adding calibrated Gaussian noise. This summarization not only improves the privacy-utility trade-off but also significantly reduces the dimensionality of the graph.

Feature Summarization. To complement the summarized structure, we construct a *compressed feature matrix* $\bar{\boldsymbol{X}}$ that preserves semantic information from the original features. Instead of perturbing raw features or training gradients—which may degrade utility or accumulate noise—we apply *objective perturbation* to a carefully designed convex optimization problem. This guarantees a one-shot DP feature transformation that maintains consistency with the summary graph structure.

Graph Reconstruction. Finally, we reconstruct a synthetic graph based on the summarized outputs. Using the learned cluster assignments, we propagate both structural and feature

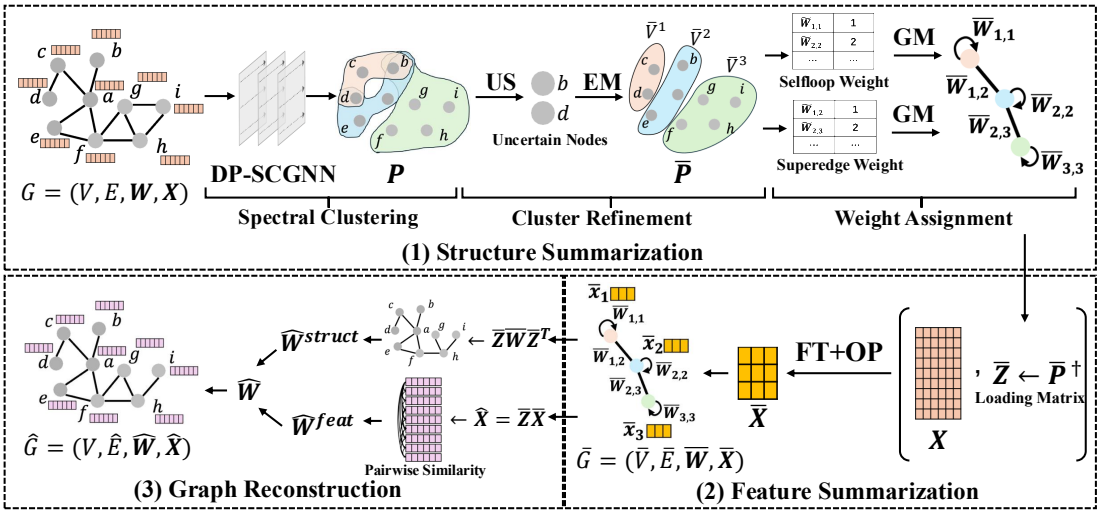


Fig. 3: DPGS framework consists of three components. (1) **Structure Summarization** clusters the input graph G using DP-SCGNN to obtain a soft assignment matrix $\boldsymbol{P}$. A subset of high-uncertainty nodes (US) is refined using the exponential mechanism to obtain a hard cluster assignment $\bar{\boldsymbol{P}}$. Gaussian noise is then added to the aggregated intra- and inter-cluster edge weights, yielding the structure summary and the weight matrix $\bar{\boldsymbol{W}}$. (2) **Feature Summarization** transforms node features via feature transformation (FT) and objective perturbation (OP) into a compressed matrix $\bar{\boldsymbol{X}}$ aligned with the structure summary, and computes the loading matrix $\bar{\boldsymbol{Z}}$ using the pseudo-inverse of $\bar{\boldsymbol{P}}$. (3) **Graph Reconstruction** reconstructs a synthetic graph $\hat{G}$ by combining structural similarity $\hat{\boldsymbol{W}}^{\text{struct}}$ and feature similarity $\hat{\boldsymbol{W}}^{\text{feat}}$ to infer edges.

information from the summary graph back to the original node space. The summary structure and features are projected to obtain node-level similarities, which are then combined to infer the presence of edges. As this reconstruction relies solely on differentially private summary representations, it incurs no additional privacy loss (by post-processing).

B. Structure Summarization

Spectral Clustering. We propose a differentially private spectral clustering module, DP-SCGNN, which jointly leverages graph topology and node features to generate clustering-aware representations. Motivated by the homophily property of real-world networks [31], our method encourages nodes with similar attributes and strong structural connectivity to be grouped together. Given a graph $G = (V, E, \boldsymbol{W}, \boldsymbol{X})$, to enable privacy-preserving representation for clustering, following the approach proposed in [32], we adopt a multi-scale GNN-based encoder with injected Gaussian noise:

$$\boldsymbol{H} = \texttt{COMBINE}\left(\left\{\text{MLP}^{(k)}(\texttt{Norm}(\boldsymbol{X}^{(k)} + \boldsymbol{\eta}^{(k)}))\right\}_{k=0}^{K}\right), \tag{1}$$

where $\boldsymbol{X}^{(k)} = \left(\boldsymbol{W}^{\top}\right)^k \boldsymbol{X}^{(0)}$ denotes the k-hop aggregated node features, and $\boldsymbol{\eta}^{(k)} \sim \mathcal{N}(0, \sigma^2 \mathbb{I})$ is the Gaussian noise added to enforce differential privacy. The operator $\texttt{Norm}$ performs ℓ_2 normalization, and $\texttt{COMBINE}$ integrates representations across different multilayer perceptron (MLP) layers.

The embedding matrix $\boldsymbol{H}$ is then fed to a multilayer perceptron with a softmax output layer to produce soft cluster assignments:

$$\boldsymbol{P} = \text{Softmax}\left(\frac{\text{MLP}_{\text{cluster}}(\boldsymbol{H}; \boldsymbol{\Theta}_{\text{MLP}})}{\tau}\right), \tag{2}$$

where $\boldsymbol{P} \in \mathbb{R}_{+}^{k \times p}$ is the assignment matrix and $\tau > 0$ is a temperature parameter controlling the sharpness of the assignments. The softmax function is applied column-wise, ensuring $P_{c,i} \in [0, 1]$ and $\boldsymbol{P}^{\top}\mathbf{1}_k = \mathbf{1}_p$, i.e., each column $\boldsymbol{P}_{:,i}$ forms a valid probability distribution over the k clusters. Moreover, the probabilistic assignments in $\boldsymbol{P}$ naturally reflect clustering uncertainty, which helps identify ambiguous (hard-to-cluster) nodes for refinement without increasing the differential privacy budget. We jointly optimize $\boldsymbol{\Theta}_{\text{GNN}}$ and $\boldsymbol{\Theta}_{\text{MLP}}$ by minimizing an unsupervised two-term loss $\mathcal{L}_u$ that approximates the relaxed MinCut objective [33]:

$$\mathcal{L}_u = \mathcal{L}_c + \mathcal{L}_o = \underbrace{-\frac{\text{tr}\left(\boldsymbol{P}^T \tilde{\boldsymbol{W}} \boldsymbol{P}\right)}{\text{tr}\left(\boldsymbol{P}^T \tilde{\boldsymbol{D}} \boldsymbol{P}\right)}}_{\mathcal{L}_c} + \underbrace{\left\| \frac{\boldsymbol{P}^T \boldsymbol{P}}{\|\boldsymbol{P}^T \boldsymbol{P}\|_F} - \frac{\boldsymbol{I}_k}{\sqrt{k}} \right\|_F}_{\mathcal{L}_o}$$

where $\tilde{\boldsymbol{W}} = \boldsymbol{D}^{-\frac{1}{2}} \boldsymbol{W} \boldsymbol{D}^{-\frac{1}{2}} \in \mathbb{R}^{p \times p}$, $\tilde{\boldsymbol{D}}$ denotes the normalized degree matrix $\boldsymbol{D}$, $\text{tr}(\cdot)$ computes the trace of a matrix, and $\|\cdot\|_F$ indicates the Frobenius norm, and k is the number of clusters. Minimizing $\mathcal{L}_c$ promotes the clustering of nodes with strong connectivity. When $\tilde{\boldsymbol{W}}_{i,j}$ is large, the inner product $\langle \boldsymbol{P}_{:,i}, \boldsymbol{P}_{:,j} \rangle$ is encouraged to be large as well, thereby promoting similar cluster assignments for connected nodes. To avoid degenerate minima of $\mathcal{L}_c$, we add an orthogonality term $\mathcal{L}_o$ that enforces near-orthogonal cluster assignments and promotes balanced cluster sizes.

Remark. To enable differentially private training of DP-SCGNN, whose loss function depends on raw graph data, we adopt the DP-SGD mechanism [34]. At each training step, the Gaussian mechanism is applied to privatize the per-example gradients before updating model parameters. However, applying DP-SGD in the GNN context typically requires

injecting substantial noise due to the interconnected nature of graph data. In the following sections, we present a formal privacy analysis and show that training the spectral clustering component satisfies differential privacy.

Cluster Refinement. To improve the reliability of cluster assignments under differential privacy, we design a refinement procedure that selectively corrects ambiguous assignments while preserving the graph's structural coherence. Instead of applying refinement to all nodes—which would lead to excessive privacy budget consumption—we focus on a small, highly uncertain subset of nodes and reassign them using the exponential mechanism. This design balances utility and privacy through targeted intervention.

As shown in Algorithm 1, we begin by measuring the uncertainty of each node's soft assignment using entropy (Lines 2-4). For node i, with assignment vector $\boldsymbol{P}_{:,i} \in \mathbb{R}^k$, we compute:

$$u_i = -\sum_{c=1}^{k} \boldsymbol{P}_{c,i} \log \boldsymbol{P}_{c,i}. \tag{3}$$

These uncertainty scores are normalized into a probability distribution $\pi_i = \frac{\exp(u_i)}{\sum_j \exp(u_j)}$, from which we sample a subset $\mathcal{R}$ of r nodes without replacement. This sampling concentrates effort on the most ambiguous nodes. For each sampled node $i \in \mathcal{R}$, we identify a small set of plausible candidate clusters by selecting the top-m entries in $\boldsymbol{P}_{:,i}$, denoted as $\mathcal{C}_i \subseteq \{1, \ldots, k\}$. To determine the most structurally appropriate cluster for node i, we define a utility score function over each candidate $c \in \mathcal{C}_i$, which counts how many of i's neighbors are confidently assigned to c:

$$s_i(c) \leftarrow \sum_{j:\boldsymbol{W}_{i,j}=1} \mathbb{I}\left[\arg\max_{c'} \boldsymbol{P}_{c',j} = c\right] \tag{4}$$

where $\{j : \boldsymbol{W}_{i,j} = 1\}$ denotes the 1-hop neighbors of node i in the adjacency matrix $\boldsymbol{W}$, and $\mathbb{I}[\cdot]$ is the indicator function (Lines 8-11). We then sample the final cluster using the exponential mechanism:

$$\mathsf{Pr}\left[c_i = c\right] \propto \exp\left(\frac{\varepsilon \cdot s_i(c)}{2}\right), \quad \text{for } c \in \mathcal{C}_i, \tag{5}$$

where ε is the privacy budget and sensitivity $\Delta_s = 1$ (Line 12). To finalize the assignment, we update $\boldsymbol{P}_{:,i}$ by retaining the original score of the selected cluster c_i and setting the remaining entries to zero (Line 13). For nodes not selected for refinement ($i \notin \mathcal{R}$), we similarly retain only the maximum entry. This enforces that the refined matrix $\bar{\boldsymbol{P}}$ is an orthogonal assignment matrix in which each column retains only one non-zero entry (Lines 15-17). This enables more interpretable downstream processing, while ensuring that the privacy budget is spent effectively.

Summary Weight Assignment. Based on the node clustering results $\bar{\boldsymbol{P}}$, our aim is to operate on a summary graph $\bar{G}$ constructed from clustered nodes. In this graph, edges are categorized into intra-supernode (within a cluster) or inter-supernode (across clusters) edges. Since intra-supernode edges often dominate, treating these two types separately mitigates noise amplification. Instead of perturbing each edge, we add Gaussian noise to aggregated edge counts: self-loop weights

Algorithm 1: CLUSTERREFINEMENT

Input: Soft assignment matrix $\boldsymbol{P} \in \mathbb{R}^{k \times p}$, adjacency matrix $\boldsymbol{W} \in \{0,1\}^{p \times p}$, refinement size r, candidate size m, privacy budget ε
Output: Refined assignment matrix $\bar{\boldsymbol{P}} \in \mathbb{R}^{k \times p}$

1 **// Uncertainty-based sampling**
2 **for** $i = 1$ **to** p **do**
3 $\quad u_i \leftarrow -\sum_{c=1}^{k} \boldsymbol{P}_{c,i} \log \boldsymbol{P}_{c,i}$
4 Normalize: $\pi_i \leftarrow \frac{\exp(u_i)}{\sum_{j=1}^{p} \exp(u_j)}$
5 Sample $\mathcal{R} \subset \{1, \ldots, p\}$ of size r from multinomial($\{\pi_i\}$) without replacement
6 **// Refine uncertain nodes**
7 Set $\bar{\boldsymbol{P}} \leftarrow \boldsymbol{0} \in \mathbb{R}^{k \times p}$
8 **for** *each* $i \in \mathcal{R}$ **do**
9 $\quad \mathcal{C}_i \leftarrow$ indices of top-m values in $\boldsymbol{P}_{:,i}$
10 $\quad$ **for** *each* $c \in \mathcal{C}_i$ **do**
11 $\quad\quad s_i(c) \leftarrow \sum_{j:\boldsymbol{W}_{i,j}=1} \mathbb{I}\left[\arg\max_{c'} \boldsymbol{P}_{c',j} = c\right]$
12 $\quad$ Sample $c_i \in \mathcal{C}_i$ with probability $\propto \exp\left(\frac{\varepsilon \cdot s_i(c)}{2}\right)$
13 $\quad$ Set $\bar{\boldsymbol{P}}_{c_i,i} \leftarrow \boldsymbol{P}_{c_i,i}$
14 **// Orthogonalize assignments**
15 **for** *each* $i \notin \mathcal{R}$ **do**
16 $\quad c_i \leftarrow \arg\max_c \boldsymbol{P}_{c,i}$
17 $\quad$ Set $\bar{\boldsymbol{P}}_{c_i,i} \leftarrow \boldsymbol{P}_{c_i,i}$
18 **return** $\bar{\boldsymbol{P}}$

for intra-supernode connections and superedge weights for inter-supernode links. This aggregation reduces sensitivity and yields a compact, noise-resilient representation of the original graph structure.

As shown in Algorithm 2, we begin by grouping nodes into supernodes based on the cluster assignment matrix $\bar{\boldsymbol{P}} \in \mathbb{R}_{+}^{k \times p}$. Each node is assigned to the cluster u with the highest membership score:

$$\bar{V}^u = \left\{ v_i \mid \arg\max_{c'} \bar{\boldsymbol{P}}_{c',i} = u \right\}, \quad \forall u \in \{1, \ldots, k\}.$$

This results in a partition of the node set into k disjoint clusters, which serve as the supernodes of the summary graph (Lines 1-2).

To define the connectivity between supernodes, we compute the normalized edge count between each pair (u, v) based on the original adjacency matrix $\boldsymbol{W} \in \{0,1\}^{p \times p}$. To ensure differential privacy, we perturb the raw edge counts using Gaussian noise. For $u \neq v$, the edge weight is defined as:

$$\bar{\boldsymbol{W}}_{u,v} = \frac{\sum_{v_i \in \bar{V}^u, v_j \in \bar{V}^v} \boldsymbol{W}_{i,j} + \mathcal{N}(0, \sigma^2)}{|\bar{V}^u| \cdot |\bar{V}^v|}.$$

For self-loops (intra-cluster connectivity), the weight is:

$$\bar{\boldsymbol{W}}_{u,u} = \frac{2\sum_{i<j,\, v_i, v_j \in \bar{V}^u} \boldsymbol{W}_{i,j} + \mathcal{N}(0, 4\sigma^2)}{|\bar{V}^u|(|\bar{V}^u| - 1)}.$$

Note that the sensitivity of counting edges is 1, since adding or removing a single edge affects at most one pairwise count (Lines 3-8).

To preserve symmetry, we mirror each entry across the diagonal: $\bar{\boldsymbol{W}}_{v,u} \leftarrow \bar{\boldsymbol{W}}_{u,v}$ (Line 9). Finally, to ensure validity under differential privacy, we clip any negative weights to zero (Line 10):

$$\bar{\boldsymbol{W}}_{u,v} \leftarrow \max(0, \bar{\boldsymbol{W}}_{u,v}).$$

The resulting matrix $\bar{\boldsymbol{W}} \in \mathbb{R}^{k \times k}$ defines a connected, undirected, and non-negative weighted summary graph that can serve as the structural backbone for downstream graph

reconstruction or representation tasks.

Algorithm 2: SUMMARYWEIGHTASSIGNMENT

Input: Adjacency matrix $\boldsymbol{W} \in \{0,1\}^{p\times p}$, cluster assignment matrix $\bar{\boldsymbol{P}} \in \mathbb{R}_{+}^{k\times p}$, Gaussian noise scale σ
Output: Noisy summary graph weight matrix $\bar{\boldsymbol{W}} \in \mathbb{R}^{k\times k}$

for $u = 1$ **to** k **do**
  $\bar{V}^u \leftarrow \{v_i \mid \arg\max_{c'} \bar{\boldsymbol{P}}_{c',i} = u\}$
for $u = 1$ **to** k **do**
  for $v = u$ **to** k **do**
    if $u \neq v$ **then**
      $\bar{\boldsymbol{W}}_{u,v} \leftarrow \frac{\sum_{v_i\in\bar{V}^u, v_j\in\bar{V}^v} \boldsymbol{W}_{i,j} + \mathcal{N}(0,\sigma^2)}{|\bar{V}^u|\cdot|\bar{V}^v|}$
    else
      $\bar{\boldsymbol{W}}_{u,u} \leftarrow \frac{2\sum_{i<j, v_i, v_j\in\bar{V}^u} \boldsymbol{W}_{i,j} + \mathcal{N}(0,4\sigma^2)}{|\bar{V}^u|(|\bar{V}^u|-1)}$
    $\bar{\boldsymbol{W}}_{v,u} \leftarrow \bar{\boldsymbol{W}}_{u,v}$; // Symmetrize
    $\bar{\boldsymbol{W}}_{u,v} \leftarrow \max(0, \bar{\boldsymbol{W}}_{u,v})$; // Normalize
return $\bar{\boldsymbol{W}}$

C. Feature Summarization

Our objective then is to derive a suitable transformation F such that $\tilde{\boldsymbol{X}} = F(\boldsymbol{X})$, i.e., it generates the summary graph's feature matrix from the original graph's features. It is essential that $\tilde{\boldsymbol{X}}$ remains similar to $\boldsymbol{X}$ with only bounded information loss. We adopt the method of Kumar et al. [35] to learn a feature transformation while bounding the information loss. This is done by minimizing the objective function:

$$\min_{\tilde{\boldsymbol{X}}} f(\tilde{\boldsymbol{X}}) = \operatorname{tr}\left(\tilde{\boldsymbol{X}}^T \boldsymbol{Z}^T \boldsymbol{L} \boldsymbol{Z} \tilde{\boldsymbol{X}}\right) + \frac{\alpha}{2}\|\boldsymbol{Z}\tilde{\boldsymbol{X}} - \boldsymbol{X}\|_F^2. \quad (6)$$

where $\boldsymbol{Z} \in \mathbb{R}^{p\times k}$ is the pseudo inverse of the assignment matrix $\boldsymbol{P}$, known as the loading matrix. Here, the first term of the objective function imposes the smoothness property on the summary graph [35], and the second term acts as a regularizer. This regularization summarizes the feature matrix of a larger graph $\boldsymbol{X} \in \mathbb{R}^{p\times n}$ into a smaller one $\tilde{\boldsymbol{X}} \in \mathbb{R}^{k\times n}$.

Objective Perturbation. According to the objective function in Eq. (6), directly solving it would involve dependencies on the raw data, such as the loading matrix $\boldsymbol{Z}$, the Laplacian matrix $\boldsymbol{L}$, and the feature matrix $\boldsymbol{X}$, all of which are derived from the original graph. It is thus necessary to consider how to incorporate a DP mechanism to learn a differentially private feature matrix $\bar{\boldsymbol{X}}$. To avoid potential structural damage caused by input perturbation and noise accumulation resulting from gradient perturbation, we perturb the objective function as

$$\min_{\bar{\boldsymbol{X}}} f(\bar{\boldsymbol{X}}) = \operatorname{tr}(\bar{\boldsymbol{X}}^T \bar{\boldsymbol{L}}_s \bar{\boldsymbol{X}}) + \frac{\alpha}{2}\|\bar{\boldsymbol{Z}}\bar{\boldsymbol{X}} - \boldsymbol{X}\|_F^2 + \operatorname{tr}(\boldsymbol{B}^T \bar{\boldsymbol{X}}). \quad (7)$$

Here, $\bar{\boldsymbol{L}}_s$ and $\bar{\boldsymbol{Z}}$ are the Laplacian matrix and loading matrix derived from the summarized graph structure $\bar{\boldsymbol{W}}$ and the private assignment matrix $\bar{\boldsymbol{P}}$, respectively. Since the structure summarization process already satisfies DP, these two matrices are used to replace the structure-related data in Eq. (6). To ensure the privacy of the feature matrix, $\operatorname{tr}(\boldsymbol{B}^T\bar{\boldsymbol{X}})$ is introduced. The third term of the private objective function adds noise into $\bar{\boldsymbol{X}}$, where $\boldsymbol{B} \sim \mathcal{N}(0, \Delta_B^2\sigma^2\mathbb{I})$ is the noise matrix. After feature normalization, we have $\Delta_B \leq \max_i \|\boldsymbol{X}_i\|^2 \leq 2k$.

Optimization. As $\bar{\boldsymbol{L}}_s$ and $\bar{\boldsymbol{Z}}^T\bar{\boldsymbol{Z}}$ are the positive semi-definite and definite matrices, the Hessian of $\bar{\boldsymbol{X}}$, i.e., $\nabla^2 f(\bar{\boldsymbol{X}}) = 2\bar{\boldsymbol{L}}_s + \alpha\bar{\boldsymbol{Z}}^T\bar{\boldsymbol{Z}}$ is a positive definite matrix. This implies that Eq. (7) defines a strongly convex optimization problem. Consequently, a closed-form solution can be obtained by setting its gradient to zero, i.e., $2\bar{\boldsymbol{L}}_s\bar{\boldsymbol{X}} + \alpha\bar{\boldsymbol{Z}}^T(\bar{\boldsymbol{Z}}\bar{\boldsymbol{X}} - \boldsymbol{X}) + \frac{1}{\alpha}\boldsymbol{B} = 0$. Then we have

$$\bar{\boldsymbol{X}} = \left(\frac{2}{\alpha}\bar{\boldsymbol{L}}_s + \bar{\boldsymbol{Z}}^T\bar{\boldsymbol{Z}}\right)^{-1}\left(\bar{\boldsymbol{Z}}^T\boldsymbol{X} - \frac{1}{\alpha}\boldsymbol{B}\right). \quad (8)$$

This optimization allows us to inject noise only once while directly solving for the optimal solution, effectively avoiding noise accumulation and preventing direct perturbation to the graph structure. Then, by integrating the summarized structure with the corresponding summarized features, the summary graph $\bar{G} = (\bar{V}, \bar{E}, \bar{\boldsymbol{W}}, \bar{\boldsymbol{X}})$ is obtained. The whole graph summarization procedure is presented in Algorithm 3, which includes three key steps, namely structure summarization (Lines 2-4), feature summarization (Lines 6-10), and returning the summary graph (Lines 11-12).

Algorithm 3: GRAPHSUMMARIZATION

Input: Adjacency matrix $\boldsymbol{W} \in \{0,1\}^{p\times p}$, feature matrix $\boldsymbol{X} \in \mathbb{R}^{p\times n}$, number of clusters k, noise scale σ, privacy budget ε
Output: Private summary graph $\bar{G} = (\bar{V}, \bar{E}, \bar{\boldsymbol{W}}, \bar{\boldsymbol{X}})$

1 **// Structure Summarization.**
2 $\boldsymbol{P} \leftarrow$ DP-SCGNN$(\boldsymbol{W}, \boldsymbol{X}, k, \varepsilon_1)$; // GNN-based soft clustering
3 $\bar{\boldsymbol{P}} \leftarrow$ CLUSTERREFINEMENT$(\boldsymbol{P}, \boldsymbol{W}, r, m, \varepsilon_2)$; // Entropy sampling + Exponential Mechanism
4 $\bar{\boldsymbol{W}} \leftarrow$ SUMMARYWEIGHTASSIGNMENT$(\boldsymbol{W}, \bar{\boldsymbol{P}}, \sigma)$; // Noisy edge aggregation
5 **// Feature Summarization.**
6 Normalize feature matrix $\boldsymbol{X}$: column-wise:$\boldsymbol{X}_{:,j} \leftarrow \boldsymbol{X}_{:,j}/\|\boldsymbol{X}_{:,j}\|_2, \quad \forall j \in \{1,\ldots,n\}$
7 Compute Laplacian matrix: $\bar{\boldsymbol{L}}_s \leftarrow$ LAPLACIAN$(\bar{\boldsymbol{W}})$;
8 Compute loading matrix: $\bar{\boldsymbol{Z}} \leftarrow \bar{\boldsymbol{P}}^{\dagger}$;
9 Generate noise: $\boldsymbol{B} \sim \mathcal{N}(0, \Delta_B^2\sigma^2\mathbb{I})$;
10 Compute the summarized feature matrix:

$$\bar{\boldsymbol{X}} = \left(\frac{2}{\alpha}\bar{\boldsymbol{L}}_s + \bar{\boldsymbol{Z}}^T\bar{\boldsymbol{Z}}\right)^{-1}\left(\bar{\boldsymbol{Z}}^T\boldsymbol{X} - \frac{1}{\alpha}\boldsymbol{B}\right)$$

// Return Summary Graph.
11 Set $\bar{V} = \{1,\ldots,k\}$, $\bar{E} = \{(u,v) \mid \bar{\boldsymbol{W}}_{u,v} > 0\}$;
12 **return** $\bar{G} = (\bar{V}, \bar{E}, \bar{\boldsymbol{W}}, \bar{\boldsymbol{X}})$;

D. Graph Reconstruction

Given the noisy summary weight matrix $\bar{\boldsymbol{W}} \in \mathbb{R}^{k\times k}$, the compressed node features $\bar{\boldsymbol{X}} \in \mathbb{R}^{k\times d}$, and the loading matrix $\bar{\boldsymbol{Z}} \in \mathbb{R}^{p\times k}$, we reconstruct the original graph structure without requiring any additional information. The key idea is to propagate structural and semantic signals from the summary graph back to the original node space, thereby recovering the adjacency matrix in a privacy-preserving manner. The workflow is shown in Algorithm 4, which involves computing structure-based similarity (Line 2), feature-based similarity (Lines 4-7), and fusion of scores (Lines 11-19), as detailed below.

Structure-based similarity. We first reconstruct pairwise structural similarity among original nodes by projecting the noisy supernode weights back to the fine-grained space:

$$\hat{\boldsymbol{W}}^{\text{struct}} = \bar{\boldsymbol{Z}}\bar{\boldsymbol{W}}\bar{\boldsymbol{Z}}^{\top}.$$

This step propagates the differentially private structure information to the original graph level.

Feature-based similarity. We then reconstruct the node features:

$$\hat{\boldsymbol{X}} = \bar{\boldsymbol{Z}}\bar{\boldsymbol{X}},$$

and compute pairwise feature similarity via cosine similarity:

$$\hat{\boldsymbol{W}}^{\text{feat}}_{i,j} = \frac{\hat{\boldsymbol{X}}_i^\top \hat{\boldsymbol{X}}_j}{\|\hat{\boldsymbol{X}}_i\| \cdot \|\hat{\boldsymbol{X}}_j\| + \xi},$$

where $\xi > 0$ is a small constant to ensure numerical stability. This captures semantic relationships between nodes that may not be reflected in the structural signal.

Fusion of scores. We combine both sources of similarity into a unified score matrix:

$$\hat{\boldsymbol{W}}^{\text{score}} = \beta \cdot \hat{\boldsymbol{W}}^{\text{struct}} + (1-\beta) \cdot \hat{\boldsymbol{W}}^{\text{feat}},$$

where $\beta \in [0,1]$ controls the trade-off between structure and features.

Then, we threshold the fused scores to obtain the reconstructed binary adjacency matrix:

$$\hat{\boldsymbol{W}}_{i,j} = \begin{cases} 1 & \text{if } \hat{\boldsymbol{W}}^{\text{score}}_{i,j} \geq \tau, \\ 0 & \text{otherwise,} \end{cases}$$

where τ is a tunable threshold hyperparameter. We define the reconstructed node set as $V = \{v_1, \ldots, v_p\}$, where p is the number of rows in $\bar{\boldsymbol{Z}}$. The predicted edge set is given by:

$$\hat{E} = \{(v_i, v_j) \mid \hat{\boldsymbol{W}}_{i,j} = 1\}.$$

This reconstruction procedure leverages only the available summary outputs, ensuring that no additional privacy budget is consumed. The integration of structure and feature signals enables accurate and utility-preserving recovery of the original graph topology.

Algorithm 4: GRAPHRECONSTRUCTION

Input: Summary edge matrix $\bar{\boldsymbol{W}} \in \mathbb{R}^{k\times k}$, summary feature matrix $\bar{\boldsymbol{X}} \in \mathbb{R}^{k\times d}$, loading matrix $\bar{\boldsymbol{Z}} \in \mathbb{R}^{p\times k}$, fusion weight $\beta \in [0,1]$, threshold τ
Output: Reconstructed graph $\hat{G}$

// Structure-based similarity
$\hat{\boldsymbol{W}}^{\text{struct}} \leftarrow \bar{\boldsymbol{Z}} \cdot \bar{\boldsymbol{W}} \cdot \bar{\boldsymbol{Z}}^\top$
// Feature-based similarity
$\hat{\boldsymbol{X}} \leftarrow \bar{\boldsymbol{Z}} \cdot \bar{\boldsymbol{X}}$
for $i = 1$ **to** p **do**
  for $j = 1$ **to** p **do**
    $\hat{\boldsymbol{W}}^{\text{feat}}_{i,j} \leftarrow \frac{\hat{\boldsymbol{X}}_i^\top \hat{\boldsymbol{X}}_j}{\|\hat{\boldsymbol{X}}_i\| \cdot \|\hat{\boldsymbol{X}}_j\| + \xi}$; `// Cosine similarity`
// Fusion
$\hat{\boldsymbol{W}}^{\text{score}} \leftarrow \beta \cdot \hat{\boldsymbol{W}}^{\text{struct}} + (1-\beta) \cdot \hat{\boldsymbol{W}}^{\text{feat}}$
// Thresholding (binary reconstruction)
$\hat{E} \leftarrow \emptyset$
for $i = 1$ **to** p **do**
  for $j = 1$ **to** p **do**
    if $\hat{\boldsymbol{W}}^{score}_{i,j} \geq \tau$ **then**
      $\hat{\boldsymbol{W}}_{i,j} \leftarrow 1$
      Add (v_i, v_j) to $\hat{E}$; `// Add predicted edge`
    else
      $\hat{\boldsymbol{W}}_{i,j} \leftarrow 0$
$V = \{v_i \mid \exists\, u \in \{1,\ldots,k\},\ \bar{\boldsymbol{P}}_{u,i} \neq 0\}$.
return $\hat{G} = (V, \hat{E}, \hat{\boldsymbol{W}}, \hat{\boldsymbol{X}})$

V. PRIVACY ANALYSIS

As shown in Fig. 3, DPGS consists of two main stages: GRAPHSUMMARIZATION and GRAPHRECONSTRUCTION. Only GRAPHSUMMARIZATION directly accesses the sensitive input graph. Therefore, it suffices to analyze the privacy of GRAPHSUMMARIZATION; the privacy guarantee for DPGS then follows from the post-processing property.

Privacy Budget Allocation. Within GRAPHSUMMARIZATION, STRUCTURESUMMARIZATION comprises three components—DP-SCGNN, cluster refinement, and summary weight assignment—which consume privacy budgets ε_{11}, ε_{12}, and ε_{13}, respectively. FEATURESUMMARIZATION consumes privacy budget ε_2. The approximate-DP parameters are allocated as follows: (i) DP-SCGNN and summary weight assignment use Gaussian noise and therefore incur failure probabilities δ_{11} and δ_{13}; (ii) cluster refinement uses the Exponential Mechanism and is pure DP (i.e., $\delta_{12} = 0$); and (iii) objective perturbation in feature summarization incurs δ_2. We set the overall failure probability as $\delta = \delta_{11} + \delta_{13} + \delta_2$.

Theorem 1 (Privacy Guarantee of DPGS)**.** *Given a graph $G = (V, E, \boldsymbol{W}, \boldsymbol{X})$, DPGS satisfies (ε, δ)*-feature-edge differential privacy*, where $\varepsilon = \varepsilon_{11} + \varepsilon_{12} + \varepsilon_{13} + \varepsilon_2, \delta = \delta_{11} + \delta_{13} + \delta_2$.*

Proof. For brevity, we provide a proof sketch; the complete proof is deferred to the appendix. We analyze privacy under the feature-edge neighboring relation (Definition 1), where two graphs differ in at most one edge and in the feature vector of at most one node.

Structure summarization. DP-SCGNN satisfies $(\varepsilon_{11}, \delta_{11})$-*feature-edge DP* by applying Gaussian noise during inference. For cluster refinement, the utility score for a candidate cluster depends on the number of neighbors assigned to that cluster, and its sensitivity under a single edge change is 1; therefore, the Exponential Mechanism yields $(\varepsilon_{12}, 0)$-*feature-edge differential privacy*. Summary weight assignment perturbs aggregated edge counts with the Gaussian mechanism, and thus satisfies $(\varepsilon_{13}, \delta_{13})$-*feature-edge differential privacy*. By sequential composition, STRUCTURESUMMARIZATION satisfies $(\varepsilon_{11}+\varepsilon_{12}+\varepsilon_{13}, \delta_{11}+\delta_{13})$-*feature-edge differential privacy*.

Feature summarization. Feature summarization accesses the raw feature matrix $\boldsymbol{X}$, and under the neighboring relation only one row of $\boldsymbol{X}$ may change. Objective perturbation adds a single Gaussian perturbation term to the convex objective, yielding $(\varepsilon_2, \delta_2)$-*feature-edge differential privacy*. *Overall mechanism.* Composing structure and feature summarization gives (ε, δ)-*feature-edge differential privacy*, where $\varepsilon = \varepsilon_{11} + \varepsilon_{12} + \varepsilon_{13} + \varepsilon_2$ and $\delta = \delta_{11} + \delta_{13} + \delta_2$.

Finally, GRAPHRECONSTRUCTION depends only on the differentially private summaries and is therefore post-processing; by post-processing invariance, it does not incur additional privacy loss. □

Privacy Accounting for DP-SCGNN. We next present the privacy accounting used for training DP-SCGNN. A key challenge in differentially private GNN training is that message passing induces dependencies, which can increase sensitivity and complicate privacy accounting. We adopt the accounting method in [36], which reduces inter-node dependency and improves the privacy-utility trade-off. The analysis is formalized using Rényi differential privacy (RDP) [37].

Theorem 2 (Privacy Guarantee for any r-Layer GNN)**.** *Let N be the number of training nodes $|V_{tr}|$, N_k be the maximum*

degree of the input graph, r be the number of GNN layers, and m be the batch size during training. For any noise standard deviation $\sigma > 0$ and clipping threshold η, each iteration t of DP-SGD satisfies (α_ρ, γ)-feature-edge Rényi differential privacy*, where*

$$\gamma = \frac{1}{\alpha_\rho - 1} \ln \mathbb{E}_\rho \left[\exp\left(\alpha_\rho(\alpha_\rho - 1) \cdot \frac{2\rho^2\eta^2}{\sigma^2} \right) \right],$$

$$\rho \sim \text{Hypergeometric}\left(N,\ \frac{N_k^{r+1} - 1}{N_k - 1},\ m \right).$$

Here, Hypergeometric *denotes the standard hypergeometric distribution [38]. By the standard composition theorem for Rényi differential privacy [37], after T iterations, DP-SCGNN training satisfies* $(\alpha_\rho, \gamma T)$-feature-edge Rényi differential privacy.

VI. Experimental Evaluation

In this section, we first introduce our experimental setup in Section VI-A, and then present the experimental results and our analysis in Section VI-B.

A. Experiment Setup

1) Datasets: Table I characterizes the 5 real-world datasets used to benchmark DPGS.

TABLE I: Summary statistics of used datasets.

Name	#Nodes	#Edges	#Features	#Classes
Cora	2708	5278	1433	7
CiteSeer	3327	9104	3703	6
PubMed	19717	44324	500	3
Coauthor	18333	163788	6805	15
DBLP	17716	105734	1639	4

2) Experimental Design: We evaluate the performance of our approach against a range of representative baselines across three categories. For **graph summarization baselines**: we include four state-of-the-art methods: Local Variation Neighborhood (LVN), proposed in [39]; SSumM, massive graph summarization method proposed in [40]; Kron, which applies Schur complement-based reduction of the Laplacian matrix [41]; and Algebraic Distance (AD), which leverages connection strength between nodes [42]. For **differentially private GNN baselines**, we compare with DP-GNN [36], DPGCN [43], GAP [32], LPGNet [44], and Eclipse [45]. Notably, all these methods are evaluated under the *edge-level* privacy setting (i.e., Edge-DP [32]). For **differentially private graph synthesis baselines**, we primarily consider methods designed to protect graph *structure* (i.e., edges) under differential privacy, while not explicitly modeling node features. Specifically, we include five representative structure-only baselines: PrivHRG [15], DER [20], TmF [19], LDPGen [22], and PrivGraph [23]. We include PrivAGS [24] as a feature-aware baseline that jointly preserves graph structure and node attributes under DP. For fair comparison, we extend PrivAGS to protect all node attributes jointly, matching the privacy setting of our feature-aware methods. We follow the recommended hyperparameters from prior work, and adapt LDPGen from local DP to the standard (central) DP model. Unless specified otherwise, we vary ε from 0.5 to 3.5 and split the total privacy budget evenly across components. We set $\delta = 1/p^2$ with $p = |V|$, and split it across approximate-DP components in GraphSummarization such that $\delta = \delta_{11} + \delta_{13} + \delta_2$ (while the Exponential Mechanism is pure DP with $\delta_{12} = 0$).

3) Evaluation Indicators: **Synthesis Quality.** To comprehensively evaluate the quality of the synthetic graphs, we adopt metrics capturing two complementary aspects of structural fidelity. For **global topological structure**, we use the *relative error (RE)* of modularity (RE_{Mod}) and of the clustering coefficient (RE_{CC}), where a lower value indicates a closer match to the original. For **node-level importance**, we rank all nodes by *eigenvector centrality (EVC)* and report the proportion of overlapping nodes among the top 1% most influential nodes ($\text{Overlap}_{\text{Node}}$) and the mean absolute error of their centrality scores (MAE_{EVC}). The formal definitions of all metrics, together with additional metrics and results, are provided in the appendix.

Downstream Task Performance. To evaluate the utility of the synthetic graphs beyond structural similarity, we assess their effectiveness on two representative downstream graph learning tasks: node classification and link prediction. These tasks reflect how well the synthetic graph supports semantic inference and connectivity modeling, and are widely used to gauge the practical value of graph representations. Strong performance in these settings indicates that the synthetic graphs preserve not only topological patterns but also task-relevant signals essential for learning meaningful representations. We use accuracy to evaluate node classification and the Area Under the ROC Curve (AUC) to assess link prediction. Accuracy quantifies the proportion of correctly predicted node labels, while AUC measures the model's ability to distinguish between positive and negative links.

B. Experimental Results and Analysis

1) Synthesis Quality: **Topological Structure.** Figures 4 and 5 report the relative error of modularity and clustering coefficients on Cora, CiteSeer, PubMed, and Coauthor datasets, capturing how well each method preserves structural properties under varying privacy budgets ε. Overall, DPGS achieves the lowest RE across all datasets and metrics, indicating superior preservation of topological features.

On CiteSeer, DPGS reduces RE of Modularity from over 0.8 (typical for TmF, DER, and LDPGen) to below 0.5 at $\varepsilon = 1.0$, with further improvements as ε increases. Similar trends are observed in clustering coefficients, where DPGS consistently outperforms all baselines across privacy levels. In contrast, methods such as TmF and PrivHRG show minimal sensitivity to ε, likely due to their direct matrix perturbation or rigid modeling assumptions.

PrivGraph does not perform as well as DPGS, particularly on CiteSeer. Although it benefits from community-level modeling, its hard clustering strategy in the community adjustment phase forces a full re-clustering of all nodes at each iteration. This introduces cumulative noise and optimization instability, often leading to suboptimal convergence and inconsistent structural recovery. These limitations are reflected in its rel-

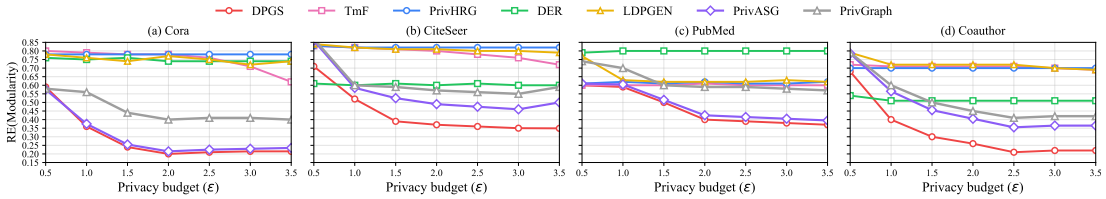


Fig. 4: Overall results on RE (Modularity).

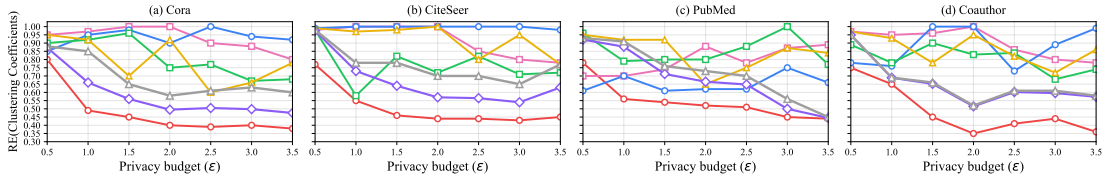


Fig. 5: Overall results on RE (Clustering Coefficients).

atively high RE in both modularity and clustering coefficient metrics, especially under small ε.

On PubMed — a larger and sparser dataset — DPGS maintains RE(Modularity) below 0.4 for $\varepsilon \geq 2$, while all other methods, including PrivGraph, exceed 0.6. Its RE on clustering coefficients is also the lowest and most stable, demonstrating robustness to both graph size and sparsity.

PrivAGS consistently outperforms PrivGraph across datasets, with marginal gains under small ε that become more pronounced as the privacy budget increases, while remaining slightly inferior to DPGS. The gap to DPGS is most evident on Coauthor and second largest on CiteSeer, indicating increased difficulty under higher feature complexity.

Node-level Importance. Figures 6 and 7 present the comparison results of node-level importance preservation using two complementary metrics: the overlap of eigenvector centrality nodes between the original and synthetic graphs, and the MAE of their EVC scores. Across four datasets, DPGS consistently outperforms all baselines once the privacy budget exceeds $\varepsilon = 1.5$. This improvement can be attributed to its summarization-based design: by clustering structurally similar nodes and performing edge reconstruction at the community level, DPGS mitigates the effect of perturbation and better maintains global spectral properties. As a result, it yields significantly higher node overlap (e.g., > 0.7 on both CiteSeer and PubMed for $\varepsilon \geq 2$) and lower MAE compared to TmF, PrivHRG, and DER.

At low privacy budgets ($\varepsilon = 0.5$), the overlap scores of DPGS remain modest as expected, since noisy clustering leads to degraded reconstruction fidelity. However, the performance significantly improves with increasing ε, while other baselines remain flat or improve marginally. This trend suggests that DPGS makes the best use of a larger budget to enhance structural recovery.

In contrast, PrivGraph and LDPGen, while competitive on certain datasets, exhibit limited adaptability. For example, PrivGraph's overlap plateaus on PubMed, likely due to its reliance on fixed statistics and hard clustering, which restrict flexibility under noise. LDPGen performs relatively well on PubMed but struggles on CiteSeer, possibly because the EVC score differences between top nodes are less pronounced, making ranking sensitive to noise.

TmF and PrivHRG, which directly perturb edge presence or rely on coarse probabilistic hierarchies, consistently underperform due to accumulated noise and limited expressiveness. DER, although data-adaptive, shows unstable results across datasets and generally falls short of DPGS's performance.

PrivAGS improves upon PrivGraph in preserving node-level importance, but remains notably weaker than DPGS when evaluated under the same privacy budget. This gap is expected, as PrivAGS was originally designed to protect a single dimension (either structure or attributes), and enforcing the same privacy strength as DPGS leads to insufficient protection of spectral information, particularly on feature-rich datasets such as Coauthor and CiteSeer.

2) Downstream Task Performance: **Node Classification.** To evaluate the utility of DPGS under different privacy constraints, we conduct node classification experiments across four benchmark datasets with privacy budgets $\varepsilon \in [0.1, 3.5]$. As reported in Table II, DPGS consistently achieves the highest accuracy across all datasets and privacy levels. Although the performance margin over strong baselines (e.g., Eclipse and LPGNet) is narrower compared to earlier settings, DPGS remains the most robust method, particularly under low ε values, where most other methods suffer significant performance degradation.

For instance, on the Cora and CiteSeer datasets with $\varepsilon = 0.1$, DPGS attains almost 67% and 64% accuracy, respectively. This demonstrates DPGS's effectiveness in preserving task-relevant information even under strict privacy constraints. On larger graphs like PubMed and Coauthor, DPGS maintains its superiority, achieving over 72% and 91% accuracy under $\varepsilon = 0.1$, respectively — outperforming all competitors.

The effectiveness of DPGS can be attributed to its two components: it first summarizes the input graph into low-sensitivity structural and semantic representations, and then applies privacy-preserving mechanisms at this intermediate level. By avoiding direct noise injection into the input graph or training gradients, DPGS significantly reduces utility loss. In contrast, baselines such as DP-GNN and DPGCN suffer from accumulated noise across multiple stages of training or inference, leading to noticeable degradation in predictive performance.

Link Prediction. Since most private learning methods are

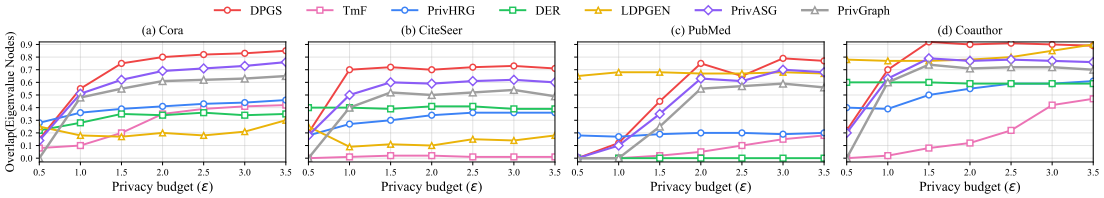


Fig. 6: Overall results on Overlap (Eigenvalue Nodes).

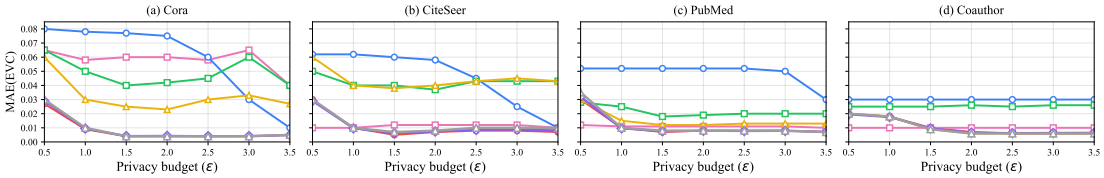


Fig. 7: Overall results on MAE (EVC).

TABLE II: Node classification performance across datasets under different ε values.

Dataset	ε	DPGCN	Eclipse	LPGNet	DP-GNN	GAP	DPGS
Cora	0.1	36.95 ± 0.42	65.11 ± 0.38	48.14 ± 0.21	27.36 ± 1.08	34.66 ± 0.25	**67.50 ± 0.20**
	3.5	70.24 ± 1.00	65.32 ± 0.61	70.50 ± 0.91	39.54 ± 0.71	57.40 ± 0.20	**69.68 ± 0.38**
CiteSeer	0.1	35.23 ± 1.06	63.07 ± 0.07	49.38 ± 0.87	28.00 ± 0.29	32.87 ± 0.47	**64.14 ± 0.15**
	3.5	61.17 ± 0.68	63.68 ± 0.48	63.20 ± 0.52	31.20 ± 0.43	56.41 ± 0.41	**67.80 ± 0.35**
PubMed	0.1	54.65 ± 0.62	71.72 ± 0.24	66.82 ± 0.88	62.24 ± 0.64	59.99 ± 0.30	**72.17 ± 0.25**
	3.5	61.06 ± 0.97	71.55 ± 0.82	74.07 ± 0.46	65.70 ± 1.13	72.01 ± 0.16	**77.73 ± 0.28**
Coauthor	0.1	63.26 ± 0.30	88.01 ± 0.14	67.46 ± 0.64	52.90 ± 0.77	80.21 ± 0.63	**90.09 ± 0.12**
	3.5	86.03 ± 0.13	89.01 ± 0.12	89.77 ± 0.10	59.01 ± 0.78	90.47 ± 0.20	**91.11 ± 0.06**

TABLE III: Link prediction performance across datasets under different summary sizes.

Dataset	Size	Ground Truth	SSumM	LVN	Kron	AD	DPGS
Cora	30%	—	68.35 ± 0.15	70.20 ± 0.58	77.10 ± 0.24	78.02 ± 0.30	**82.42 ± 0.04**
	10%	84.31 ± 0.75	66.08 ± 0.56	67.82 ± 0.22	75.13 ± 0.13	77.02 ± 0.36	**80.21 ± 0.13**
	5%	—	63.32 ± 1.05	63.00 ± 0.95	73.28 ± 0.24	75.05 ± 0.13	**77.89 ± 0.03**
CiteSeer	30%	—	72.05 ± 0.47	71.45 ± 0.25	73.01 ± 0.14	75.13 ± 0.17	**76.86 ± 0.03**
	10%	78.62 ± 0.62	69.71 ± 0.38	69.53 ± 0.58	69.83 ± 0.36	73.88 ± 0.30	**75.66 ± 0.03**
	5%	—	62.89 ± 0.82	63.93 ± 0.64	68.13 ± 0.39	72.00 ± 0.45	**72.29 ± 0.04**
PubMed	5%	—	61.30 ± 0.83	62.10 ± 0.90	67.05 ± 0.40	77.12 ± 0.33	**81.19 ± 0.04**
	3%	83.28 ± 0.57	60.66 ± 0.58	62.13 ± 0.57	66.28 ± 0.22	72.30 ± 0.23	**75.85 ± 0.06**
	1%	—	57.35 ± 0.58	61.60 ± 0.23	66.22 ± 0.38	68.15 ± 0.45	**72.17 ± 0.02**

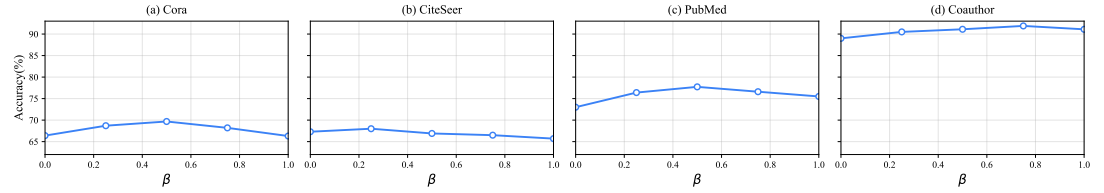


Fig. 8: Overall results on β tuning.

primarily designed for node classification, we evaluate the effectiveness of DPGS on the link prediction task, comparing it with several summarization-based baselines, including SSumM, LVN, Kron, and AD. Following established practice, we vary the summarization ratio from 5% to 30% for Cora and CiteSeer, and from 1% to 5% for the larger PubMed dataset to assess scalability under compression.

As shown in Table III, DPGS consistently achieves the highest AUC across all datasets and summarization levels. On Cora, for instance, DPGS reaches an AUC of 82.42% at a 30% summary size and maintains a strong lead even when compressed to 5%. Similarly, on CiteSeer and PubMed, DPGS outperforms all baselines by a notable margin, especially under extreme compression (e.g., 1% summary on PubMed), where it still achieves 72.17% AUC compared to AD's 68.15%.

These improvements can be attributed to DPGS's capability of jointly summarizing both structural patterns and node features into a compact representation with reduced sensitivity. Privacy-preserving noise is applied only once during the summarization phase, and no additional noise is introduced during reconstruction. In contrast, competing methods either disregard node attributes (e.g., LVN, Kron) or apply heuristics that lead to cumulative distortions. DPGS effectively balances utility and privacy, especially under aggressive compression, making it more robust and scalable in practice.

3) Robustness against MIA: We evaluate the resilience of DPGS to membership inference attacks (MIA) under the setting of [46], where the adversary aims to determine whether

TABLE IV: MIA performance across datasets.

Dataset	Method	$\varepsilon = 0.1$	$\varepsilon = 0.5$	$\varepsilon = 1$	$\varepsilon = 3$
Cora	GAP	50.12 ± 0.03	50.23 ± 0.04	50.42 ± 0.02	51.10 ± 0.04
	DP-GNN	50.24 ± 0.04	50.25 ± 0.02	50.20 ± 0.01	50.22 ± 0.03
	DPGS	50.18 ± 0.02	50.22 ± 0.02	50.39 ± 0.03	50.61 ± 0.04
CiteSeer	GAP	50.02 ± 0.09	50.18 ± 0.03	50.30 ± 0.05	51.58 ± 0.01
	DP-GNN	50.06 ± 0.02	50.05 ± 0.02	50.62 ± 0.04	50.67 ± 0.05
	DPGS	50.08 ± 0.03	50.08 ± 0.02	50.28 ± 0.03	50.46 ± 0.05
PubMed	GAP	50.08 ± 0.05	50.35 ± 0.04	51.12 ± 0.02	51.29 ± 0.03
	DP-GNN	50.06 ± 0.03	50.06 ± 0.04	50.35 ± 0.02	50.66 ± 0.02
	DPGS	50.29 ± 0.02	50.30 ± 0.02	50.50 ± 0.06	50.96 ± 0.02
Coauthor	GAP	50.03 ± 0.05	50.18 ± 0.04	50.52 ± 0.05	51.48 ± 0.02
	DP-GNN	50.00 ± 0.02	50.08 ± 0.04	50.57 ± 0.02	50.93 ± 0.06
	DPGS	50.23 ± 0.02	50.30 ± 0.07	50.33 ± 0.04	50.44 ± 0.05

a given node was part of the target model's training set. Such attacks typically exploit overfitting by using confidence scores from a shadow model trained on a similar distribution. As shown in Table IV, all methods yield AUC values close to 50%, indicating that the attack model performs no better than random guessing. Notably, DPGS maintains consistently low AUC values across all datasets and privacy levels, with minimal variation as ε increases. This demonstrates that DPGS effectively limits information leakage, offering strong membership privacy even under relaxed privacy budgets.

4) The impact of β: We further investigate the impact of varying the weighting parameter β in the score matrix under a fixed privacy budget of $\varepsilon = 3.5$. The results shown in Figure 8 exhibit a consistent *bell-shaped trend* across all datasets: accuracy increases from relatively low values at $\beta = 0$, reaches a peak at an intermediate value of β, and subsequently decreases as β approaches 1. This observation confirms that an appropriate balance between structural and feature information is crucial for maximizing utility under differential privacy.

Several dataset-specific patterns can be observed. On **Cora**, the peak occurs at $\beta = 0.5$, indicating that structural and feature information contribute almost equally to the utility. In contrast, **CiteSeer** achieves its highest accuracy when β is closer to the feature side ($\beta = 0.2$), which aligns with the higher quality of its node features. This demonstrates the importance of feature information in this dataset. **PubMed** reaches its maximum accuracy around $\beta = 0.55$, suggesting a slightly stronger reliance on structural information, while still benefiting from feature signals. Finally, **Coauthor** achieves the highest performance when β is shifted toward the structural side ($\beta = 0.8$), reflecting the dataset's well-defined community structures. Importantly, at $\beta = 1$, Coauthor still maintains the best performance among all datasets, highlighting the robustness of structural information for this graph. Guided by the above analysis, we fix $\beta = 0.5$ in all experiments, which strikes a balance among fairness, privacy, and consistency while preserving utility.

VII. Related Work

Differentially Private Graph Neural Networks. Incorporating differential privacy (DP) into Graph Neural Networks (GNNs) is increasingly important due to the sensitivity of graph data. Existing methods enforce edge- or node-level DP during training. GAP [32] perturbs neighborhood aggregation with calibrated noise, while DPDGC [47] decouples convolution into structural and feature components for tighter sensitivity control under DP-SGD. ProGAP [48] progressively injects noise to improve stability and privacy–utility tradeoffs. LPGNet [44] adjusts message passing to protect edges with less noise, and Eclipse [45] reduces sensitivity via low-rank SVD on the adjacency matrix before perturbation. LinkTeller [43] further shows that many defenses remain vulnerable to inference attacks, highlighting the need for stronger protection and better threat models.

Privacy Attacks and Threat Models on Graphs. Graph data is highly vulnerable to privacy attacks due to its rich relational structure. Adversaries may infer hidden edges via structure-based attacks, which exploit topological patterns [49], [26], or feature-based attacks, which rely solely on node attributes [50], [51]. De-anonymization attacks [52], [53] further attempt to re-identify nodes using auxiliary data and matching algorithms. DPGS mitigates such risks by disrupting correlations between observed signals and sensitive edges, reducing susceptibility to both structure- and feature-based attacks. Nevertheless, attacks outside the DP framework remain an open challenge, calling for alternative privacy notions and structural obfuscation techniques.

Graph Summarization and Coarsening. Graph summarization and coarsening reduce graph size while preserving key properties, enabling efficient storage [54], scalable analytics [27], and visualization [55]. Traditional methods rely on heuristics, whereas recent approaches adopt information-theoretic or optimization-based formulations. SSumM [40] uses the Minimum Description Length (MDL) principle to merge nodes and sparsify edges, achieving compression with fidelity guarantees. Kumar et al. [35] coarsen both structure and features by optimizing a feature-aware similarity objective, preserving semantic consistency for downstream tasks. These techniques not only improve efficiency but also complement DP by reducing data sensitivity before noise injection, making them well-suited for privacy-preserving graph analytics.

VIII. Conclusion

We propose DPGS, a novel framework for differentially private graph synthesis that integrates structure and feature summarization to enable accurate and privacy-preserving graph generation. By applying differential privacy during the summarization phase, DPGS significantly reduces sensitivity and avoids noise injection in downstream tasks. Our framework supports structure-feature fusion and uses a one-shot perturbation strategy to achieve strong utility under strict privacy budgets. Extensive experiments on node classification, link prediction, and membership inference demonstrate that DPGS consistently outperforms state-of-the-art baselines in both performance and privacy. These results validate the effectiveness of summarization-based synthesis as a promising paradigm for private graph publishing.

APPENDIX

Proof of Theorem 1

Proof: We analyze the privacy of DPGS by decomposing GRAPHSUMMARIZATION into its components and applying sequential composition and post-processing under the *feature-edge neighboring* relation (Definition 1), i.e., neighboring graphs differ in at most one edge and in the feature vector of at most one node.

(1) Structure Summarization. *DP-SCGNN (inference-stage privacy).* In DPGS, DP-SCGNN is used to produce the soft assignment matrix $\boldsymbol{P}$ for structure summarization (Algorithm 3, Line 2). Following Eq. (1), DP-SCGNN injects Gaussian noise into the multi-hop aggregated features:

$$\boldsymbol{H} = \texttt{COMBINE}\Big(\big\{\text{MLP}^{(k)}(\texttt{Norm}(\boldsymbol{X}^{(k)} + \boldsymbol{\eta}^{(k)}))\big\}_{k=0}^{K}\Big),$$
$$\boldsymbol{\eta}^{(k)} \sim \mathcal{N}(0, \sigma^2\mathbb{I}),$$

and then generates soft cluster assignments via Eq. (2):

$$\boldsymbol{P} = \text{Softmax}\left(\frac{\text{MLP}_{\text{cluster}}(\boldsymbol{H}; \boldsymbol{\Theta}_{\text{MLP}})}{\tau}\right)$$

In this privacy analysis, we focus on the *inference-stage* privacy guarantee of DP-SCGNN, i.e., the privacy of its released output $\boldsymbol{P}$ with respect to changes in the input graph G. Under the feature-edge neighboring relation, the sensitivity of the aggregation output is bounded, and the Gaussian perturbation in Eq. (2) ensures that the released clustering representation (and hence $\boldsymbol{P}$) satisfies $(\varepsilon_{11}, \delta_{11})$-*feature-edge differential privacy*.

Cluster refinement. For each uncertain node $i \in \mathcal{R}$, cluster refinement uses the Exponential Mechanism (Algorithm 1, Line 12) to sample a final cluster:

$$\Pr[c_i = c] \propto \exp\left(\frac{\varepsilon_{12} \cdot s_i(c)}{2\Delta_s}\right),$$

where $s_i(c)$ counts the number of neighbors of node i assigned to cluster c (Eq. (4)). Under the feature-edge neighboring relation, changing one node feature vector does not affect $s_i(c)$, while changing a single edge changes $s_i(c)$ by at most 1, i.e., $\Delta_s = 1$. Therefore, cluster refinement satisfies $(\varepsilon_{12}, 0)$-*feature-edge differential privacy*.

Summary weight assignment. Summary weight assignment computes the summary edge matrix $\bar{\boldsymbol{W}}$ by aggregating intra-/inter-cluster edges and adding Gaussian noise (Algorithm 2, Lines 3–8):

$$\bar{\boldsymbol{W}}_{u,v} = \frac{\sum_{v_i \in \bar{V}^u,\, v_j \in \bar{V}^v} \boldsymbol{W}_{i,j} + \mathcal{N}(0, \sigma^2)}{|\bar{V}^u| \cdot |\bar{V}^v|},$$
$$\bar{\boldsymbol{W}}_{u,u} = \frac{2\sum_{i<j,\, v_i, v_j \in \bar{V}^u} \boldsymbol{W}_{i,j} + \mathcal{N}(0, 4\sigma^2)}{|\bar{V}^u|(|\bar{V}^u| - 1)}.$$

Under a single-edge change, the aggregated count changes by at most 1, yielding bounded sensitivity. Since this step depends only on the adjacency matrix and the (private) clustering result, it satisfies $(\varepsilon_{13}, \delta_{13})$-*feature-edge differential privacy*.

By sequential composition, STRUCTURESUMMARIZATION satisfies

$(\varepsilon_{11} + \varepsilon_{12} + \varepsilon_{13},\ \delta_{11} + \delta_{13})$-feature-edge differential privacy.

(2) Feature Summarization. Feature summarization computes $\bar{\boldsymbol{X}}$ via objective perturbation in Eq. (7):

$$\min_{\bar{\boldsymbol{X}}} \text{tr}(\bar{\boldsymbol{X}}^\top \bar{\boldsymbol{L}}_s \bar{\boldsymbol{X}}) + \frac{\alpha}{2}\|\bar{\boldsymbol{Z}}\bar{\boldsymbol{X}} - \boldsymbol{X}\|_F^2 + \text{tr}(\boldsymbol{B}^\top \bar{\boldsymbol{X}}),$$
$$\boldsymbol{B} \sim \mathcal{N}(0, \Delta_B^2 \sigma^2 \mathbb{I}).$$

Here, $\bar{\boldsymbol{L}}_s$ and $\bar{\boldsymbol{Z}}$ are derived from the differentially private output of STRUCTURESUMMARIZATION (i.e., $\bar{\boldsymbol{W}}$ and $\bar{\boldsymbol{P}}$) and can thus be treated as fixed with respect to the raw input graph. Under the feature-edge neighboring relation, the only remaining sensitive part is the raw feature matrix $\boldsymbol{X}$, where at most one row can change arbitrarily. Therefore, objective perturbation yields $(\varepsilon_2, \delta_2)$-*feature-edge differential privacy*.

(3) Composition and Post-processing. By sequential composition of STRUCTURESUMMARIZATION and FEATURESUMMARIZATION, GRAPHSUMMARIZATION satisfies

$$\big(\varepsilon_{11} + \varepsilon_{12} + \varepsilon_{13} + \varepsilon_2,\ \delta_{11} + \delta_{13} + \delta_2\big)$$

feature-edge differential privacy. Finally, GRAPHRECONSTRUCTION (Algorithm 4) only accesses the differentially private summaries (e.g., $\bar{\boldsymbol{W}}$, $\bar{\boldsymbol{X}}$, and $\bar{\boldsymbol{Z}}$) and thus incurs no additional privacy loss by post-processing invariance. Hence, the overall DPGS mechanism satisfies (ε, δ)-*feature-edge differential privacy* with $\varepsilon = \varepsilon_{11}+\varepsilon_{12}+\varepsilon_{13}+\varepsilon_2$ and $\delta = \delta_{11}+\delta_{13}+\delta_2$. ■

Proof of Theorem 2

We provide a detailed proof for the following theorem:

Theorem 3 (Privacy Guarantee for any r-Layer GNN)**.** *Let N be the number of training nodes V_{tr}, N_k be the maximum degree of the input graph, r be the number of GNN layers, and m be the batch size during training. For any choice of the noise standard deviation $\sigma > 0$ and clipping threshold η, each iteration t of DP-SGD satisfies* (α_ρ, γ)-feature-edge Rényi differential privacy*, where*

$$\gamma = \frac{1}{\alpha_\rho - 1} \ln \mathbb{E}_\rho\left[\exp\left(\alpha_\rho(\alpha_\rho - 1) \cdot \frac{2\rho^2\eta^2}{\sigma^2}\right)\right],$$
$$\rho \sim \text{Hypergeometric}\left(N,\ \frac{N_k^{r+1} - 1}{N_k - 1},\ m\right).$$

Here, Hypergeometric *denotes the standard hypergeometric distribution [38]. By the standard composition theorem for Rényi Differential Privacy [37], after T iterations, the GNN training procedure satisfies* $(\alpha_\rho, \gamma T)$-feature-edge Rényi differential privacy.

Proof. Let G and G' be two *feature-edge adjacent* graphs, i.e., they differ by (i) a change in a single edge and/or (ii) an arbitrary change in the feature vector of a single node. Consider an r-layer message-passing GNN trained with DP-SGD over the training node set V_{tr} of size N.

Affected training nodes. In an r-layer GNN, the representation (and hence the loss/gradient) of a training node depends only on its r-hop computation graph. Under the assumption that the (computation) degree is upper bounded by N_k, the number of training nodes whose r-hop computation graphs can be affected by a single feature change (at one node) or a single edge change is bounded by the size of an $(r+1)$-depth N_k-ary tree:

$$d \ \le\ \frac{N_k^{r+1} - 1}{N_k - 1}.$$

Let $\mathcal{A} \subseteq V_{tr}$ denote this (worst-case) set of potentially affected training nodes, with $|\mathcal{A}| \leq d$.

Mini-batch overlap. At iteration t, DP-SGD samples a mini-batch $\mathcal{B}_t$ of size m uniformly without replacement from V_{tr}. Let $\rho := |\mathcal{A} \cap \mathcal{B}_t|$ be the number of affected training nodes appearing in the batch. Then

$$\rho \sim \text{Hypergeometric}(N,\ d,\ m)\,.$$

Sensitivity of the summed clipped gradient. Let each per-node gradient be clipped to ℓ_2-norm at most η. Between G and G', only the ρ affected examples in $\mathcal{B}_t$ may change, and each such clipped gradient can change by at most 2η in ℓ_2-norm. Therefore, the ℓ_2-sensitivity of the summed gradient $\mathbf{u}_t$ is upper bounded by

$$\Delta(\mathbf{u}_t) \ \leq\ 2\rho\eta.$$

Per-iteration feature-edge RDP. DP-SGD releases $\tilde{\mathbf{u}}_t = \mathbf{u}_t + \mathcal{N}(0, \sigma^2\mathbb{I})$. Conditioned on ρ, this is a Gaussian mechanism with sensitivity $2\rho\eta$, which by the standard RDP bound for the Gaussian mechanism (e.g., [37]) satisfies

$$(\alpha_\rho, \gamma_\rho)\text{-RDP}, \qquad \gamma_\rho = \frac{\alpha_\rho (2\rho\eta)^2}{2\sigma^2} = \alpha_\rho \cdot \frac{2\rho^2\eta^2}{\sigma^2}.$$

Since ρ is random due to mini-batch sampling, the unconditional mechanism is a mixture over ρ. Using the log-moment (MGF) form of RDP, we obtain

$$\gamma = \frac{1}{\alpha_\rho - 1} \ln \mathbb{E}_\rho \left[\exp\left(\alpha_\rho(\alpha_\rho - 1) \cdot \frac{2\rho^2\eta^2}{\sigma^2} \right) \right],$$

which matches the theorem statement. Hence, each iteration t satisfies (α_ρ, γ)-*feature-edge RDP*.

Composition over T iterations. By the standard composition theorem for Rényi DP [37], after T iterations the training procedure satisfies $(\alpha_\rho, \gamma T)$-*feature-edge RDP*. □

Evaluation Metric Definitions

We provide the formal definitions of the synthesis-quality metrics summarized in Section VI. The relative errors (RE) of modularity and clustering coefficient are computed as

$$\text{RE}_{\text{Mod}} = \frac{|\hat{Q} - Q|}{\max(\xi, Q)}, \quad \text{RE}_{\text{CC}} = \frac{|\hat{Y} - Y|}{\max(\xi, Y)},$$

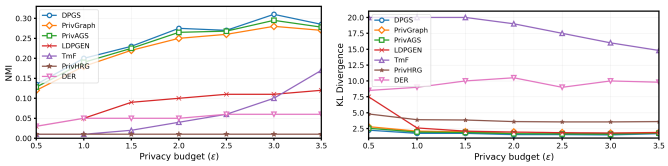
where Q and $\hat{Q}$ denote the modularity of the original and synthetic graphs, Y and $\hat{Y}$ denote their average clustering coefficients, and ξ is a small constant to prevent division by zero. To evaluate node importance preservation, we rank all nodes by eigenvector centrality (EVC) and compute two comparison metrics: (1) the proportion of overlapping nodes among the top 1% most influential nodes, and (2) the mean absolute error (MAE) of their centrality scores:

$$\text{Overlap}_{\text{Node}} = \frac{|N \cap \hat{N}|}{|N|}, \quad \text{MAE}_{\text{EVC}} = \frac{1}{T}\sum_{i=1}^{T} |\hat{s}_i - s_i|,$$

where N and $\hat{N}$ are the sets of top 1% nodes ranked by EVC in the original and synthetic graphs, and s_i, $\hat{s}_i$ are their corresponding EVC scores.

Additional Experimental Results

We evaluate the quality of the synthetic graphs using two additional metrics: Normalized Mutual Information (NMI) and Kullback–Leibler (KL) divergence. NMI quantifies the alignment between community detection results on the original and



NMI on Cora KL Divergence on Cora

Fig. 9: Results on NMI and KL Divergence over dataset Cora

synthetic graphs, while KL divergence measures the difference in degree distributions between the two graphs. Below are the formal definitions of both metrics.

a) Normalized Mutual Information (NMI).: Let $A = \{A_1, A_2, \ldots, A_R\}$ and $B = \{B_1, B_2, \ldots, B_S\}$ be two partitions of a graph $G = (V, E)$. Their overlap can be represented using a contingency table H, where H_{ij} denotes the number of nodes shared by clusters A_i and B_j. Let x_i (resp. y_j) be the sum of the i-th row (resp. j-th column) in H. The NMI between A and B is computed as:

$$\text{NMI}(A, B) = \frac{-2\sum_{i=1}^{R}\sum_{j=1}^{S} H_{ij} \log\left(\frac{H_{ij} n}{x_i y_j}\right)}{\sum_{i=1}^{R} x_i \log\left(\frac{x_i}{n}\right) + \sum_{j=1}^{S} y_j \log\left(\frac{y_j}{n}\right)}. \quad (9)$$

b) Kullback–Leibler (KL) Divergence.: Given the degree distributions $P(x)$ and $\hat{P}(x)$ of the original and synthetic graphs respectively, the KL divergence is defined as:

$$D_{\text{KL}}(P \parallel \hat{P}) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{\hat{P}(x)}\right). \quad (10)$$

The results are shown in Figure 9. For NMI, DPGS consistently achieves the highest scores across privacy budgets, with a clear margin under strict privacy. This is mainly because DPGS applies graph summarization (supernodes/superedges) to reduce dimensionality and global sensitivity, thus requiring less injected noise for a fixed ε. In addition, DPGS leverages a GNN for feature-aware clustering initialization while using a spectral clustering module to mitigate DP-induced dependence and prevent noise amplification. PrivGraph improves with ε but remains below DPGS, suggesting higher sensitivity to perturbation. PrivAGS shows a monotonic improvement trend and outperforms PrivGraph, yet stays slightly inferior to DPGS.

For KL divergence, DPGS produces the lowest divergence and remains stable across ε, indicating better preservation of the original degree distribution. Beyond sensitivity reduction from summarization, the refinement stage employs the Exponential Mechanism for private selection, which reduces noise accumulation and stabilizes convergence. In contrast, TmF and DER maintain relatively large divergence and exhibit weaker gains from increasing ε, implying more distorted global statistics. PrivGraph and PrivAGS benefit from larger ε but still lag behind DPGS, consistent with DPGS's differentiated perturbation for dense and sparse structures and the final post-processing consistency enforcement. In summary, DPGS remains robust under strict privacy budgets and achieves a better balance between noise resilience and information retention, consistently outperforming all baselines on both community structure alignment and degree distribution matching.

REFERENCES

[1] J. Leskovec, D. Huttenlocher, and J. Kleinberg, "Signed Networks in Social Media," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2010, pp. 1361–1370.

[2] J. Leskovec, J. Kleinberg, and C. Faloutsos, "Graph Evolution: Densification and Shrinking Diameters," *ACM transactions on Knowledge Discovery from Data (TKDD)*, vol. 1, no. 1, pp. 2–es, 2007.

[3] C. Gao, Y. Zheng, N. Li, Y. Li, Y. Qin, J. Piao, Y. Quan, J. Chang, D. Jin, X. He *et al.*, "A survey of graph neural networks for recommender systems: Challenges, methods, and directions," *ACM Transactions on Recommender Systems*, vol. 1, no. 1, pp. 1–51, 2023.

[4] D. Sharma, R. Shukla, A. K. Giri, and S. Kumar, "A Brief Review on Search Engine Optimization," in *2019 9th International Conference on Cloud Computing, Data Science & Engineering*. IEEE, 2019, pp. 687–692.

[5] F. Jiang, C. K. Leung, and A. G. Pazdor, "Big Data Mining of Social Networks for Friend Recommendation," in *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 2016, pp. 921–922.

[6] Y. Wang, L. Xie, B. Zheng, and K. C. Lee, "High Utility K-Anonymization for Social Network Publishing," *Knowledge and Information Systems*, vol. 41, no. 3, pp. 697–725, 2014.

[7] J. Cheng, A. W.-c. Fu, and J. Liu, "K-Isomorphism: Privacy Preserving Network Publication against Structural Attacks," in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, 2010, pp. 459–470.

[8] S. Ji, P. Mittal, and R. Beyah, "Graph Data Anonymization, De-Anonymization Attacks, and De-Anonymizability Quantification: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 1305–1326, 2016.

[9] J. Qian, X.-Y. Li, C. Zhang, L. Chen, T. Jung, and J. Han, "Social Network De-Anonymization and Privacy Inference with Knowledge Graph Model," *IEEE Transactions on Dependable and Secure Computing*, vol. 16, no. 4, pp. 679–692, 2017.

[10] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating Noise to Sensitivity in Private Data Analysis," in *Theory of Cryptography Conference*. Springer, 2006, pp. 265–284.

[11] L. Du, Z. Zhang, S. Bai, C. Liu, S. Ji, P. Cheng, and J. Chen, "AHEAD: Adaptive Hierarchical Decomposition for Range Query under Local Differential Privacy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1266–1288.

[12] Z. Zhang, T. Wang, N. Li, S. He, and J. Chen, "CALM: Consistent Adaptive Local Marginal for Marginal Release under Local Differential Privacy," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 212–229.

[13] T. Wang, J. Q. Chen, Z. Zhang, D. Su, Y. Cheng, Z. Li, N. Li, and S. Jha, "Continuous Release of Data Streams under both Centralized and Local Differential Privacy," in *ACM CCS*, 2021.

[14] A. Sala, X. Zhao, C. Wilson, H. Zheng, and B. Y. Zhao, "Sharing Graphs Using Differentially Private Graph Models," in *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference*, 2011, pp. 81–98.

[15] Q. Xiao, R. Chen, and K.-L. Tan, "Differentially Private Network Data Release via Structural Inference," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 911–920.

[16] M. Hay, C. Li, G. Miklau, and D. Jensen, "Accurate Estimation of the Degree Distribution of Private Networks," in *2009 Ninth IEEE International Conference on Data Mining*. IEEE, 2009, pp. 169–178.

[17] H. Sun, X. Xiao, I. Khalil, Y. Yang, Z. Qin, H. Wang, and T. Yu, "Analyzing Subgraph Statistics from Extended Local Views with Decentralized Differential Privacy," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019.

[18] T. Ji, C. Luo, Y. Guo, Q. Wang, L. Yu, and P. Li, "Community Detection in Online Social Networks: A Differentially Private and Parsimonious Approach," *IEEE transactions on Computational Social Systems*, vol. 7, no. 1, pp. 151–163, 2020.

[19] H. H. Nguyen, A. Imine, and M. Rusinowitch, "Differentially Private Publication of Social Graphs at Linear Cost," in *2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 2015, pp. 596–599.

[20] R. Chen, B. Fung, P. S. Yu, and B. C. Desai, "Correlated Network Data Publication via Differential Privacy," *The VLDB Journal*, vol. 23, no. 4, pp. 653–676, 2014.

[21] A. Clauset, C. Moore, and M. E. Newman, "Hierarchical Structure and the Prediction of Missing Links in Networks," *Nature*, vol. 453, no. 7191, pp. 98–101, 2008.

[22] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating Synthetic Decentralized Social Graphs with Local Differential Privacy," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 425–438.

[23] Q. Yuan, Z. Zhang, L. Du, M. Chen, P. Cheng, and M. Sun, "{PrivGraph}: differentially private graph data publication by exploiting community information," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3241–3258.

[24] S. Ye, L. Chen, Z. Zhang, Y. Gao, Y. Wang, and X. Xu, "Privags: Differentially private attributed graph synthesis," *Proceedings of the ACM on Management of Data*, vol. 3, no. 6, pp. 1–25, 2025.

[25] Y. Liu, T. Safavi, A. Dighe, and D. Koutra, "Graph summarization methods and applications: A survey," *ACM computing surveys (CSUR)*, vol. 51, no. 3, pp. 1–34, 2018.

[26] X. Xian, T. Wu, Y. Liu, W. Wang, C. Wang, G. Xu, and Y. Xiao, "Towards Link Inference Attack against Network Structure Perturbation," *Knowledge-Based Systems*, vol. 218, p. 106674, 2021.

[27] M. Riondato, D. García-Soriano, and F. Bonchi, "Graph summarization with quality guarantees," *DMKD*, vol. 31, no. 2, pp. 314–349, 2017.

[28] K. LeFevre and E. Terzi, "Grass: Graph structure summarization," in *SDM*, 2010.

[29] C. Dwork, A. Roth *et al.*, "The Algorithmic Foundations of Differential Privacy," *Foundations and Trends® in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[30] F. McSherry and K. Talwar, "Mechanism Design via Differential Privacy," in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS'07)*. IEEE, 2007, pp. 94–103.

[31] M. McPherson, L. Smith-Lovin, and J. M. Cook, "Birds of a feather: Homophily in social networks," *Annual review of sociology*, vol. 27, no. 1, pp. 415–444, 2001.

[32] S. Sajadmanesh, A. S. Shamsabadi, A. Bellet, and D. Gatica-Perez, "{GAP}: Differentially private graph neural networks with aggregation perturbation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3223–3240.

[33] I. S. Dhillon, Y. Guan, and B. Kulis, "Kernel k-means: spectral clustering and normalized cuts," in *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, 2004, pp. 551–556.

[34] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 308–318.

[35] M. Kumar, A. Sharma, S. Saxena, and S. Kumar, "Featured graph coarsening with similarity guarantees," in *International Conference on Machine Learning*. PMLR, 2023, pp. 17 953–17 975.

[36] A. Daigavane, G. Madan, A. Sinha, A. G. Thakurta, G. Aggarwal, and P. Jain, "Node-level differentially private graph neural networks," *arXiv preprint arXiv:2111.15521*, 2021.

[37] I. Mironov, "Rényi differential privacy," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 2017, pp. 263–275.

[38] C. Forbes, M. Evans, N. Hastings, and B. Peacock, *Statistical Distributions*. Wiley, 2011. [Online]. Available: https://books.google.co.in/books?id=YhF1osrQ4psC

[39] A. Loukas and P. Vandergheynst, "Spectrally approximating large graphs with smaller graphs," in *International conference on machine learning*. PMLR, 2018, pp. 3237–3246.

[40] K. Lee, H. Jo, J. Ko, S. Lim, and K. Shin, "Ssumm: Sparse summarization of massive graphs," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 144–154.

[41] F. Dorfler and F. Bullo, "Kron reduction of graphs with applications to electrical networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 1, pp. 150–163, 2012.

[42] J. Chen and I. Safro, "Algebraic distance on graphs," *SIAM Journal on Scientific Computing*, vol. 33, no. 6, pp. 3468–3490, 2011.

[43] F. Wu, Y. Long, C. Zhang, and B. Li, "Linkteller: Recovering private edges from graph neural networks via influence analysis," in *2022 ieee symposium on security and privacy (sp)*. IEEE, 2022, pp. 2005–2024.

[44] A. Kolluri, T. Baluta, B. Hooi, and P. Saxena, "Lpgnet: Link private graph networks for node classification," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1813–1827.

[45] T. Tang, Y. Niu, S. Avestimehr, and M. Annavaram, "Edge private graph neural networks with singular value perturbation," *arXiv preprint arXiv:2403.10995*, 2024.

[46] I. E. Olatunji, W. Nejdl, and M. Khosla, "Membership inference attack on graph neural networks," in *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. IEEE, 2021, pp. 11–20.

[47] E. Chien, W.-N. Chen, C. Pan, P. Li, A. Ozgur, and O. Milenkovic, "Differentially private decoupled graph convolutions for multigranular topology protection," *Advances in Neural Information Processing Systems*, vol. 36, pp. 45 381–45 401, 2023.
[48] S. Sajadmanesh and D. Gatica-Perez, "Progap: Progressive graph neural networks with differential privacy guarantees," in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 2024, pp. 596–605.
[49] Y. Zhang, M. Humbert, B. Surma, P. Manoharan, J. Vreeken, and M. Backes, "Towards Plausible Graph Anonymization," in *Proceedings of the 27th Annual Network and Distributed System Security Symposium*, 2020.
[50] C. Yang, L. Zhong, L.-J. Li, and L. Jie, "Bi-Directional Joint Inference for User Links and Attributes on Large Social Graphs," in *Proceedings of the 26th International Conference on World Wide Web Companion*, 2017, pp. 564–573.
[51] N. Eagle, A. Pentland, and D. Lazer, "Inferring Friendship Network Structure by Using Mobile Phone Data," *Proceedings of the National Academy of Sciences*, vol. 106, no. 36, pp. 15 274–15 278, 2009.
[52] W.-H. Lee, C. Liu, S. Ji, P. Mittal, and R. B. Lee, "How to Quantify Graph De-Anonymization Risks," in *Information Systems Security and Privacy*, 2017, pp. 84–104.
[53] Y. Zhao and I. Wagner, "Using Metrics Suites to Improve the Measurement of Privacy in Graphs," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 259–274, 2020.
[54] H. Toivonen, F. Zhou, A. Hartikainen, and A. Hinkka, "Compression of weighted graphs," in *KDD*, 2011.
[55] D. Koutra, U. Kang, J. Vreeken, and C. Faloutsos, "Vog: Summarizing and understanding large graphs," in *SDM*, 2014.